

A Theoretical Study of DBLog: Certified Virtual Cuts for a Snapshot-Equivalent Replay of Live Databases

Andreas Andreakis

Abstract

DBLog is a change-data-capture (CDC) mechanism for copying a table or selected keys from a source database while continuing to stream new changes from its commit log. It reads a table in primary-key ordered chunks and brackets each read with low and high watermarks in the source log. A chunk row is discarded if a log event for the same key appears anywhere in that window. The remaining rows are emitted as refresh events after the high watermark. DBLog allows backfills to run at any time during normal operation, rather than only as an initial bootstrap, while writes and live capture continue. It was introduced in the 2019 Netflix Tech Blog post [1] and further discussed in the 2020 DBLog paper [2]. The mechanism has since been adopted by open-source projects, including Debezium [3] and Apache Flink CDC [4].

The original blog post and paper explained this mechanism but did not provide a formal correctness proof or a precise description of its replay result. In this paper, we formalize how chunk reads and log events are combined and prove that their replay reconstructs the source state up to a specific log position for the keys being copied. We define this as a **virtual cut**, representing a snapshot-equivalent replay without requiring a physical snapshot. DBLog emits the events needed to reconstruct the source state downstream, without storing that state itself.

To verify executions independently, we provide a certificate that validates this equivalence from recorded log evidence and chunk observations. Once chunk processing completes and covers all keys, this guarantee extends to the whole table. Furthermore, appending subsequent change-stream events advances the cut forward, formalizing the seamless transition from historical backfill to live streaming. All definitions and proofs are mechanized in Isabelle/HOL.

1 Introduction

DBLog, introduced in the Netflix Tech Blog in 2019 [1] and described in the 2020 DBLog paper [2], is a change-data-capture (CDC) mechanism for backfilling a downstream system from a source database while continuing to stream new changes. DBLog reads the table in primary-key chunks and brackets each chunk read between low and high watermarks in the source log. The watermarks bracket the read without knowing its exact source-log position. DBLog then discards any chunk row whose key collides with a CDC event anywhere

within that window to ensure that refresh events don't override more recent CDC events. CDC events retain their source order, and the remaining chunk rows are emitted as refresh events after the high watermark. Variants of this mechanism are now used by Debezium [3] and Apache Flink CDC [4], where the backfill phase is called an *incremental snapshot*.

The mechanism was designed to allow backfills at any time, not only initially, without interrupting the flow of change events. A backfill can run during normal operation, cover an entire table or selected keys, and pause or resume between chunks. This lets operators control the scope and pace of the work without stalling live capture.

Because chunks are read at distinct source-log positions, intermediate writes at the source can render individual row observations stale. The collected chunk reads do not form a single physical snapshot. We establish that DBLog's combination of chunk refreshes and change events nonetheless reconstructs the exact per-key state of the source at a designated commit-log frontier, and we prove how that equivalence advances forward as the source commits subsequent writes.

While the original 2020 paper explained how the mechanism works operationally, it did not formalize the reconstructed database state or provide a proof. We supply that foundation by formalizing how chunk observations and captured changes are combined and proving that their replay reconstructs the source state. We characterize this result through the concept of a **virtual cut**. A **virtual cut** formalizes snapshot equivalence: replaying the sequence of chunk observations and streaming writes gathered during a backfill produces the exact state of the source database at a specific commit-log coordinate, for every key in the targeted scope. The cut is *virtual* because rows were read at different times without taking a physical snapshot. A **certified virtual cut** packages this claim into an inspectable certificate, allowing an independent checker to verify that the replay matches the source from recorded log watermarks and chunk reads.

The proof is organized in five layers. Layers 0 and 1 define source histories, replay events, and abstract DBLog runs characterized by a scope (the targeted set of primary keys) and a frontier (the designated commit-log coordinate), alongside the conditions for a wellformed run. Layer 2 proves that every wellformed run produces a virtual cut at that frontier on that scope. Layer 3 lifts this guarantee to inspectable certificates, showing that an independent checker can verify the cut directly from recorded log evidence. Layer 4 derives full whole-table equality, establishing the coverage conditions needed

to prove that all existing rows and concurrent writes across the table are accounted for. The watermark mechanics enter as assumptions, including the single-coordinate attribution of each chunk read described in Section 3.2. They are not derived by the abstract model. Figure 1 summarizes these dependencies. Every theorem is machine-checked in the companion Isabelle/HOL development, archived at <https://doi.org/10.5281/zenodo.21732790>.

The initial certified cut is not the end of DBLog operation. Once the chunk loop completes, DBLog continues consuming ordinary CDC events. We prove that appending exactly the source events for keys in scope between frontiers f and f' advances a cut at f to a cut at f' . We also prove that a cut on one scope remains valid when restricted to a smaller scope. Normalization of overlapping or retried chunks, and sink state correctness remain future work.

The theorems assume that the source log is retained through the claimed frontier, each chunk read captures its rows at a single commit-log position, and the CDC stream delivers all intermediate change events in committed order. A certificate checker verifies that this recorded evidence is internally consistent, but cannot determine whether an external connector observed the physical database accurately. These guarantees are strictly source-side and scoped to the backfilled keys, with whole-table equality following under the Layer 4 conditions. They do not assert downstream sink convergence or exactly-once delivery.

Section 2 develops a running example and Section 3 reviews the mechanism used by the model. Section 4 summarizes the claims and limitations. Sections 5 through 10 define the formal objects and prove the core results. Section 11 proves continuation and restriction. Section 12 separates source-side replay correctness from destination-state convergence. Sections 13 through 15 cover related work, future extensions, and verification methodology.

We use five status labels throughout (Table 1).

Each theorem states its assumptions directly. These labels distinguish the machine-checked result from conditions supplied by a deployment or its source connector.

2 Bootstrapping and Advancing a Virtual Cut with a Running Example

We begin with a concrete backfill before introducing the formal model. The example shows what DBLog reads while the source accepts concurrent writes, how those observations form a replay, and why that replay can match the source without being a physical snapshot. It also shows how the matching point advances when the source commits another event.

We use this example throughout the paper. This section

presents the mechanism operationally, and Section 5 introduces the corresponding definitions.

2.1 Setup

The source database contains an account table

```
accounts(account_id, balance)
```

initially populated with two rows:

```
account_id = 100 balance = 5000 (Alice's account, $50)
account_id = 200 balance = 10000 (Bob's account, $100)
```

A third user, Carol, has not yet opened an account. Therefore, account_id 300 does not exist in the table.

During the backfill, application writes continue while DBLog reads chunks and watches the source log. We construct a replay whose per-account outcome matches the source at a chosen frontier, although the chunk reads occur at different source-log coordinates.

2.2 What the Source Log Records During the Backfill Window

While DBLog is working, four writes commit in the source log in the following order:

```
c1 Update 100 → 3000 (Alice's balance changes to $30)
c2 Insert 300 → 2500 (Carol opens an account at $25)
c3 Update 200 → 12000 (Bob's balance changes to $120)
c4 Update 300 → 3500 (Carol's balance changes to $35)
```

Each c_i denotes a source-log coordinate representing a position in the source database's commit order, where $c_1 < c_2 < c_3 < c_4$.

After c_4 commits, the source state across these three accounts is

```
account 100 balance = 3000 (after the c1 update)
account 200 balance = 12000 (after the c3 update)
account 300 balance = 3500 (after the c4 update)
```

The assembled replay must reconstruct this per-account state.

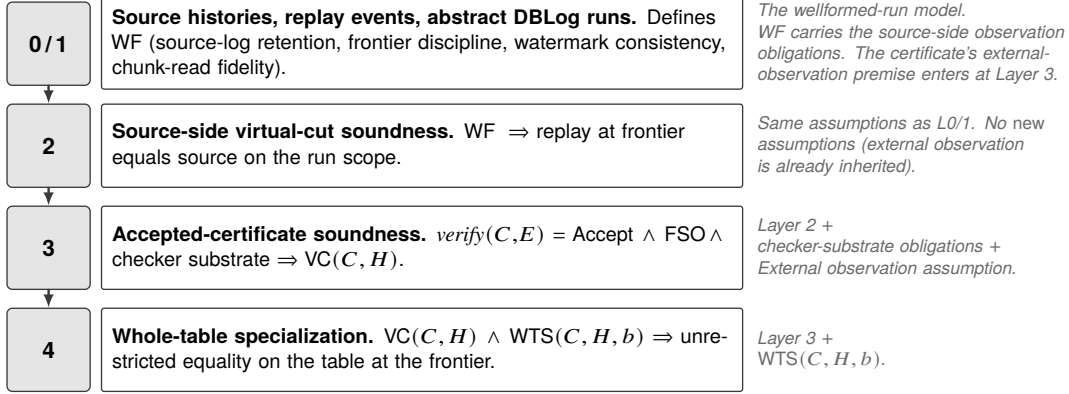
2.3 What the Chunks Read

DBLog reads the table while source writes continue. It partitions the table into two **chunks** by primary key range:

- **Chunk A** covers $account_id \in \{100, 200\}$.
- **Chunk B** covers $account_id \in \{300\}$.

The two chunk ranges are disjoint, and together they cover every account the backfill claims. Key partitioning is configured by the deployment rather than inferred by DBLog. Chunks can also cover keys that do not yet exist in the table. For instance, chunk B covers account 300 before Carol opens it.

The theorem ladder: source-side core (Layers 0–4)



Post-core extension map: independent extensions, *not* ladder rungs

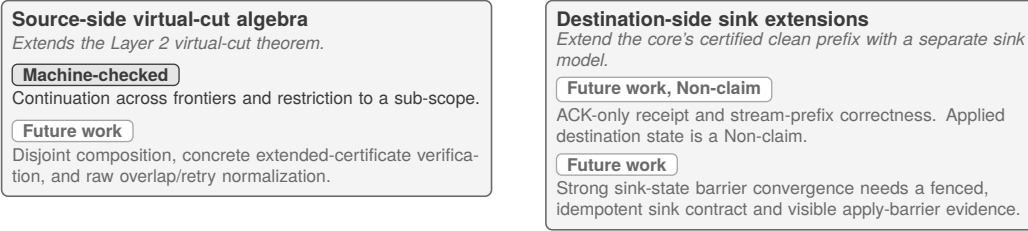


Figure 1: Theorem ladder and extensions. Layers 0–4 form the source-side core, with each layer using the premises of the layers above it. Continuation and scope restriction extend the Layer 2 virtual-cut result. Disjoint composition, extended-certificate verification, and overlap/retry normalization remain Future work. Destination-state results require a separate sink model. A cut obtained from an accepted certificate retains the certificate assumptions and faithful-source observation premise.

Each chunk is read at its own moment in source-log time. The deployment must ensure that a multi-row chunk read reflects a single source-log coordinate, an obligation made precise in Section 3.2.

Chunk A runs at coordinate c_1 , observing the state immediately after c_1 's update committed:

```
chunk A read at c1:
  account 100 balance = 3000 (already updated by c1)
  account 200 balance = 10000 (no write to 200 yet)
```

Chunk B runs later, at coordinate c_3 , observing Carol's row after Carol's c_2 insert but before Carol's c_4 update:

```
chunk B read at c3:
  account 300 balance = 2500 (Carol's initial balance from c2)
```

The two chunks run at different source-log moments. No single physical snapshot of the table has been taken. Chunk A saw Bob at \$100. By the time chunk B was reading, Bob's balance had already moved to \$120.

Although both chunk reads observe live rows here, a chunk read can also record a key as *absent*. If chunk B had executed before Carol's c_2 insert, it would have observed account 300 as absent, and that recorded absence is replayed identically to a concrete value.

2.4 The Chosen Point of Equivalence

The backfill targets a specific source-log moment for the assembled replay to match. We designate **frontier** $f = c_4$, representing the source-log position immediately after the last write in the sequence. The **scope** of the backfill comprises the claimed accounts, $K = \{100, 200, 300\}$ (Alice, Bob, Carol).

Deterministic replay of the gathered events equals the source state at f on K , although no chunk is read at f , the three accounts are never read together, and source writes continue throughout.

2.5 How the Model Assembles the Replay

The model treats every observed source-log event as an ordinary **CDC event** (the captured record of one committed source write, such as an insert, update, or delete at its source-log coordinate), every chunk-read row as a **refresh event**, and concatenates them in source-log order. Such an event is a *refresh* because it re-states a key's current stored value, read directly from the table, rather than reporting an incremental change as a CDC event does.

This raw replay retains every chunk refresh at its attributed

Table 1: Status vocabulary used throughout the paper.

Status	Meaning in this paper
Machine-checked	The claim is proved in the companion Isabelle/HOL development, under the assumptions stated where the claim appears. The paper gives the proof idea or proof explanation in the body.
Deployment obligation	The operator or implementation must establish this condition in a deployment.
External observation assumption	The source-side observation is assumed accurate (satisfying FSO). The model cannot prove it from the certificate alone.
Future work	A named extension or theorem path that this paper does not claim.
Non-claim	A property that the paper does not prove.

read coordinate. Section 3.2 explains its relationship to DBLog’s window-based suppression and output order. For our example the assembled replay sequence is:

- | | |
|------------------------------------|--|
| 1. CDC at c_1 : | Update account 100 $\rightarrow$ 3000 |
| 2. Refresh from chunk A at c_1 : | account 100 observed as 3000 |
| 3. Refresh from chunk A at c_1 : | account 200 observed as 10000 |
| 4. CDC at c_2 : | Insert account 300 $\rightarrow$ 2500 |
| 5. CDC at c_3 : | Update account 200 $\rightarrow$ 12000 |
| 6. Refresh from chunk B at c_3 : | account 300 observed as 2500 |
| 7. CDC at c_4 : | Update account 300 $\rightarrow$ 3500 |

Both refreshes from chunk A sit at c_1 . The refresh from chunk B sits at c_3 . CDC events sit at the source-log coordinate they were committed at. Where a CDC event and a chunk refresh share a coordinate (as item 1 and items 2–3 all sit at c_1), the CDC event is placed first, because a chunk read at a coordinate already reflects every write committed at that coordinate. Figure 2 places these chunk reads and CDC events on the source-log timeline.

This is the **raw replay** form. It shows the chunk refreshes DBLog gathered even when a later CDC event for the same key supersedes them. The normalized clean prefix may omit refreshes dominated by later CDC events, such as item 3 for account 200 and item 6 for account 300. Applying either the raw or the normalized form yields the same per-key state. The raw sequence shows what DBLog observed, while the normalized sequence makes stale-refresh suppression explicit for the formal model.

To reconstruct per-account state, we play the seven raw items left to right, each one writing the per-account map. When two items concern the same account, the later one wins for that account’s value.

Tracing the replay sequence in order:

- Item 1 sets account 100 to 3000.
- Item 2, chunk A’s refresh of account 100, confirms 100 = 3000. No change.
- Item 3, chunk A’s refresh of account 200, sets account 200 to 10000.
- Item 4 introduces account 300 at 2500.
- Item 5 sets account 200 to 12000. This overrides chunk A’s earlier observation: chunk A observed balance \$100 at c_1 , which the later log update at c_3 (\$120) supersedes.
- Item 6, chunk B’s refresh of account 300, confirms 300 = 2500. No change yet.

- Item 7 sets account 300 to 3500. This overrides chunk B’s earlier observation: chunk B observed \$25 at c_3 , which the update at c_4 (\$35) likewise overrides.

The final per-account state, restricted to scope $K = \{100, 200, 300\}$, is

account 100	balance = 3000
account 200	balance = 12000
account 300	balance = 3500

The source state at $f = c_4$ restricted to K is

account 100	balance = 3000
account 200	balance = 12000
account 300	balance = 3500

They match. The assembled replay reproduces the source state at f across all accounts in scope K , even though chunk A and chunk B were read at distinct earlier coordinates, and even though both chunks contained rows that had become stale before reaching f .

2.6 Advancing the Cut to a Later Frontier

The seven-item replay above forms a virtual cut at frontier $f = c_4$ on scope $K = \{100, 200, 300\}$, matching the source state at that point. Backfill operations are not restricted to this initial cut. Once all chunks in the plan have been processed (here, chunks A and B), chunk scanning terminates. However, as the source continues committing writes, DBLog keeps consuming the source log, allowing the cut to advance.

Suppose another write commits after c_4 :

c_5 Delete 200 (*Bob closes the account*)

DBLog does not re-read a chunk to account for this change. The single CDC event committed after c_4 and through c_5 is the *continuation segment* for that interval. Appending that segment advances the cut, provided that the CDC feed delivers it accurately and the source log still retains the interval. Section 11 states these conditions precisely. The replay sequence gains one event:

8. CDC at c_5 : Delete account 200

Playing item 8 after the seven-item replay makes account 200 absent. As with an insert or update, the later source-log event overrides both the earlier refresh and the c_3 update. For a delete, the resulting state is absence.

The eight-item replay is now a virtual cut at the later frontier $f' = c_5$. Its per-account state on K is account 100 = 3000, account 200 absent, account 300 = 3500, which is the source state at c_5 on K . The point of equivalence moved from c_4 to c_5 by appending the one event the source committed in between. No chunk was re-read. Had that event been an ordinary insert or update rather than a delete, it would have appended in exactly the same way. This is the running example’s instance of the continuation result of Section 11. Ongoing DBLog execution thus produces an evolving sequence of virtual cuts, where each successive cut advances the prior state by appending committed source changes.

This says nothing, by itself, about whether a destination consuming the replay has applied item 8 or reached the state in which account 200 is absent. That remains a separate sink-side question (Section 12).

2.7 What “Snapshot-Equivalent” Means

At no point in this example did DBLog read the entire table at one source-log position. Chunk A saw account 100 and account 200 at c_1 . Chunk B saw account 300 at c_3 . Because the two reads happened at different points in the source database’s commit order, DBLog must reconcile them through the source log rather than read them as one instantaneous observation.

Instead, DBLog captures sufficient source-log evidence around each chunk read to determine which observations remain authoritative at the chosen frontier and which have been superseded. The assembled replay sequence combines chunk refreshes and CDC events deterministically to yield the exact per-key state of the source at frontier f on scope K .

In this paper, *snapshot-equivalent replay* means that the replay outcome equals the source state at f restricted to K . It does not mean that DBLog performed a physical snapshot read.

2.8 Scope of the Example

The example covers one three-key backfill. It establishes replay equality for that scope, but it does not extend to the following properties:

- **It is not a physical cut.** DBLog never performed a single read covering all of {100, 200, 300}. Chunk A read accounts 100 and 200 at c_1 , and chunk B read account 300 at c_3 . The equivalence is *extensional*. Outcomes match at f on K , although DBLog did not take a snapshot.
- **It is not a whole-table claim.** The example claims only the three accounts in K . Other accounts in the source table are not addressed by this example. Whole-table correctness

is a derived specialization handled later in the paper, with extra conditions attached.

- **It is not a sink-state claim.** What we have shown is the assembled replay sequence. Whether a sink that consumes this sequence converges to the same per-account state is an additional contract on the consumer, including idempotency, ordering, and fencing. The example does not establish that contract.

2.9 Required Coordination

Avoiding a physical snapshot read does not remove the need for coordination. The example relies on four source-side conditions:

- **A source-log ordering** that is monotone and well-defined enough to compare the chunk-read coordinate of chunk A (c_1) to the CDC event for account 200 at c_3 and conclude that c_3 is later. Without a comparable ordering on commit positions, the “later wins” rule cannot be applied.
- **Chunk-read watermarks** placed in the source log around each chunk read, allowing DBLog to locate chunk A’s read in source-log order. The watermark bracket places its observation of account 200 before the c_3 event for account 200, identifying the event as strictly newer. The watermark bracket bounds the read’s position in source-log order, as described in the next section.
- **Source-log retention.** The source log must retain each required event until DBLog has captured it. If the c_3 update to account 200 were lost before it was captured, Bob’s stale chunk value would survive and the replay would not match the source at f .
- **Chunk-read consistency.** Chunk A’s two rows must both reflect the source at the single coordinate c_1 . The watermark bracket locates the read in source-log order but does not by itself establish that single-coordinate attribution. The deployment must establish this property, through a consistent statement-level read or through per-row evidence that all returned row images match one common source coordinate. Section 3.2 makes this obligation precise.

The deployment must establish these conditions. The formal model records them as explicit predicates and assumptions.

2.10 Connection to the Formal Model

Section 3 defines the operational terms used above, including chunks, watermarks, refresh events, CDC events, replay prefixes, frontiers, and scopes. Sections 5 through 11 then give these terms formal definitions and prove the general results illustrated here. The seven-item replay matches the source at $f = c_4$ on K . Appending the eighth item advances the same equality to $f' = c_5$.

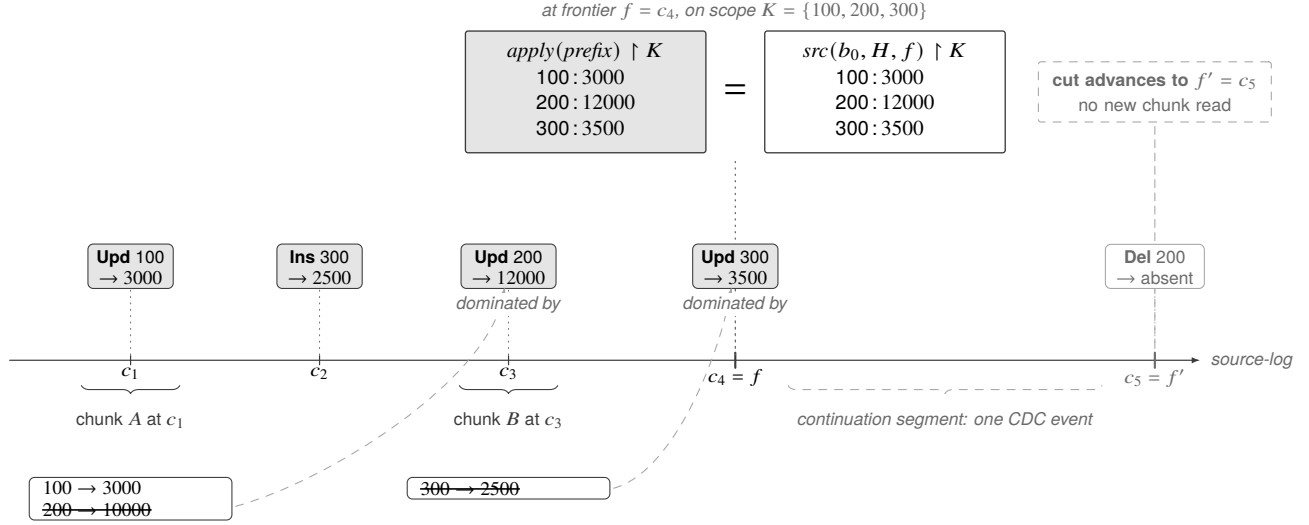


Figure 2: Two chunk reads at different source-log coordinates assemble into a replay whose state at frontier $f = c_4$ equals the source on scope $K = \{100, 200, 300\}$. Stale observations (strikethrough) are dominated by later source-log writes. No single physical read gathers every key in scope, so the cut is virtual rather than physical. That is the bootstrap cut at c_4 . The lighter elements to the right show the continuation of Section 2.6. One further source-log event (c_5 , a delete of account 200) is appended as the continuation segment for the half-open interval $(c_4, c_5]$. It begins strictly after the frontier c_4 and requires no additional chunk read. This advances the same cut to the later frontier $f' = c_5$.

3 The DBLog Mechanism

The running example used two chunks and four source-log events. We now define the operational parts of the general mechanism. Figure 3 shows the example on a source-log axis, including the chunk reads, refresh windows, and merged raw replay.

3.1 Chunks and Scope

DBLog begins with a **scope**, which is the set of primary keys for which the backfill will make a claim. In the example, the scope was the three account ids $\{100, 200, 300\}$. In a larger deployment the scope might be a table, a shard, a tenant, or a selected key range.

The scope is partitioned into **chunks**. The chunks are pairwise disjoint and together cover the claimed scope. Each key in the scope therefore belongs to exactly one chunk. The deployment chooses the chunk ranges, and the model takes that plan as an input. A chunk may also cover keys that do not yet exist in the table. An observed key absence within a chunk range is recorded as evidence just like an existing row.

Unique ownership is required by the current model. If two chunks own the same key, or if no chunk owns a key in scope, the replay lacks a uniquely defined refresh source for that key. Overlapping and retried chunks require a normalization step and are left as a future extension.

The model also abstracts away how the finite chunk plan is discovered. A deployment must still know when the chunk

loop has reached the intended end of the claimed key scope. For a whole-table run, a common way to establish that fact is to record the table’s maximum primary key, or an equivalent end-of-table observation, with enough source-side context to know which table population the bound refers to. For a selected range, shard, tenant, or other scoped run, the analogous requirement is an end-of-scope marker for that chosen scope. The deployment must provide this evidence. The model separately assumes that the primary-key order and the source observation used for the bound are accurate.

3.2 Watermarks and Chunk-Read Coordinates

Watermarks bound the source-log interval containing a chunk read. DBLog uses this interval without needing to identify the read’s exact position within it. Before a chunk read, DBLog writes a *low watermark* to a dedicated table in the source. After the scan, it writes a *high watermark*. Both writes appear in the source log at definite coordinates, with the chunk SELECT between them. The baseline mechanism therefore needs write access to the source. A read-only deployment instead relies on existing source-log coordinates, such as transaction identifiers or LSN markers.

The watermark pair establishes the read’s *provenance* in source-log order. The formal model attributes the returned rows to one *chunk-read coordinate*, such as c_1 for chunk A in the running example. Every row image is treated as matching the source at that coordinate.

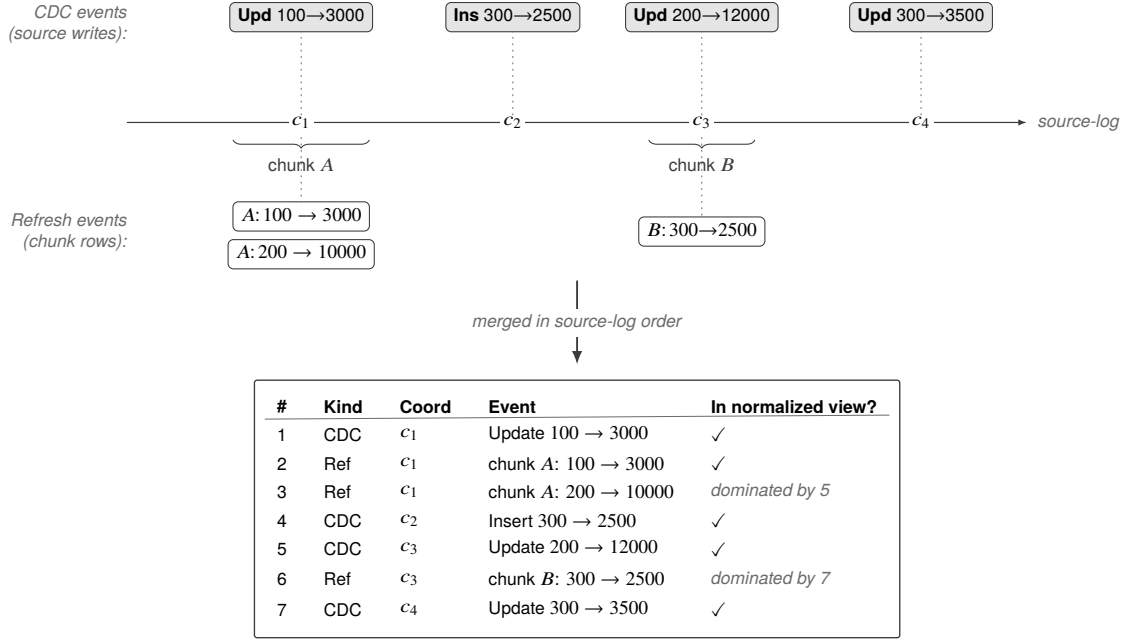


Figure 3: The DBLog mechanism in the running example. Source writes become *CDC events* above the axis. Chunk-read rows become *refresh events* (below the axis, grouped by chunk and bracketed by source-log watermarks). The two streams merge in source-log order into the *raw clean prefix* below, which is the list used by the Layer 2 theorem. A refresh row whose key is touched by a later source write is marked *dominated*. The raw clean prefix keeps it, and an optional normalized view may drop it without changing the *apply* result (§3.4).

The watermark bracket alone does not establish this single-coordinate attribution. A multi-row SELECT is not instantaneous. The deployment must also provide either read semantics that make all rows consistent at one source coordinate, such as a statement-level snapshot or repeatable-read scan, or per-row evidence that all returned row images match one common source coordinate. Thus, “chunk A read at c_1 ” combines two distinct conditions: the watermarks bracket the read around c_1 , and the returned row images accurately reflect the source state at c_1 .

In the original DBLog mechanism, the interval between the two watermarks is also the window over which chunk rows with matching CDC events are suppressed. A chunk row is discarded when the source log shows a write for the same key inside that window, whether that write committed before or after the chunk read. The model represents that behavior with two simpler parts. Each returned row has a valid chunk-read coordinate, and replay applies a frontier-wide *latest-event-wins* rule (§3.3). The raw replay keeps every chunk row.

For a completed watermark window whose CDC events have been captured in source order, both formulations reach the same per-key state at the window close. For a key in the chunk’s domain, there are three cases. If a same-key write commits after the chunk read but within the window, the last such write determines the key’s final state in both formulations.

If the only same-key writes inside the window commit before or at the chunk read, the read already reflects them. The chunk row then carries the same value or absence as the last of those writes, and discarding it changes nothing. If no same-key write falls inside the window, the row survives in both formulations. Original DBLog emits it after the high watermark, while the model places it at the chunk-read coordinate. No write to that key lies between the two positions, so the per-key result is the same. A write after the high watermark overrides the row in both formulations.

The Isabelle development mechanizes the first case as the dominated-refresh equivalence (§3.4). The other two cases use refresh fidelity, a wellformed-run obligation, together with source-state and *apply* lemmas the development proves. It does not state the window-discard transformation as a separate theorem.

The formal run retains the low and high watermarks as evidence for the chunk-read coordinate. They do not add another state-equality condition. Section 5.5 records the required source-log ordering, retention, coordinate provenance, and row-image accuracy. The last condition appears there as “Refresh fidelity.”

DBLog avoids a single physical snapshot read. It does not avoid source-log coordination.

3.3 Refresh Events and Ordinary CDC Events

The model represents each chunk-read row as a **refresh event**. A refresh records that key k held a given value, or was absent, at the chunk-read coordinate. A source-log write becomes an ordinary CDC event that records an insert, update, or delete at its source-log coordinate.

Both kinds of event enter one replay vocabulary. Ordinary CDC events carry committed source writes. Refresh events carry row observations from chunk reads. Applying a replay means walking the event sequence left to right and updating the per-key state. Later events for the same key win. That behavior is what made Bob's c_3 update override chunk A's older refresh in the example.

3.4 Raw Replay and Normalized Clean Prefix

The model constructs a finite sequence called the *clean prefix*. We use two forms of this sequence. The theorems use the *raw* form. The *normalized* form is an optional, *apply*-equivalent presentation.

The **raw replay** is the evidence-preserving sequence. It may include a refresh even if a later CDC event for the same key dominates it. Raw replay is useful operationally because it shows exactly what DBLog observed.

The **normalized clean prefix** makes stale-refresh suppression explicit. It may omit refresh events that are dominated by newer source-log evidence, provided suppressing them does not change the final *apply* state. In the example, Bob's chunk-A refresh was dominated by Bob's later c_3 update. Carol's chunk-B refresh was dominated by Carol's later c_4 update. Carrying or suppressing those dominated refreshes gives the same final per-key state. The underlying formal connection is state equivalence over the claimed scope. Suppressing dominated refreshes for the same key preserves the replay outcome under *apply* for all claimed keys.

Dominated-refresh equivalence. *A refresh dominated by a same-key CDC event at a strictly later source coordinate may be retained or suppressed without changing apply on any claimed key.* This fact connects the raw and normalized prefixes. It also covers the later-write case of original DBLog's in-window suppression. The case of a write before or at the chunk read is argued separately in §3.2. Section 7 states the Layer 2 theorem about the raw replay prefix and uses this equivalence to extend its conclusion to the optional normalized presentation.

The raw replay and its optional normalization satisfy three properties:

- no required CDC event is missing.
- no stale refresh is allowed to resurrect an older value past a newer event.
- suppressing a refresh is justified only when newer source-log evidence dominates it for the same key.

3.5 Frontier and Clean Prefix

The **frontier** is the source-log coordinate at which a cut is claimed (concretely, a MySQL executed global-transaction-identifier (GTID) set, a PostgreSQL log sequence number (LSN), or a CockroachDB resolved timestamp). A certificate claims equality at a particular frontier, not at an unspecified eventual state. The clean prefix contains the replay evidence needed through that frontier for the claimed scope.

Operationally, a DBLog certificate asserts that for a given scope K and frontier f , replaying the certified prefix reconstructs the exact source state at f on K .

The rest of the paper makes that sentence precise. Section 5 defines the source and replay models. Section 6 names the equality as a virtual cut. Section 7 proves the source-side theorem for wellformed runs. Sections 8 and 9 move from runs to accepted certificates.

The clean prefix is finite but not necessarily small. It contains a refresh for each key in the claimed scope and every in-scope CDC event through the frontier, so it scales with the scope and the change volume over the certified interval. For a large table backfilled over a busy window, the clean prefix is correspondingly large. The normalized clean prefix trims refreshes that newer evidence dominates. The raw form, on which the theorems are stated, does not.

3.6 Advancing the Frontier Beyond the Initial Backfill

Once the chunk loop covers the claimed scope and terminates, DBLog continues consuming ordinary CDC events as the source commits them. The source-side result does not need a fresh chunk read for every later frontier. If the backfill has established a virtual cut at f , appending exactly the in-scope source events committed after f and through a later frontier f' , in source order, advances that cut to f' .

The continuation theorem formalizes this step in Section 11. Once the backfill establishes an initial virtual cut, subsequent CDC stream events advance it. The chunk reads happen once. Later cuts are derived by appending valid source-log change events. This step relies on ordered source-log coordinates and retention of the events between f and f' . Section 11 states the result and its assumptions.

DBLog combines a finite chunk plan, source-log watermarks, chunk refreshes, and ordinary CDC events into an equivalent replay at a named frontier. The deployment must provide an ordered and retained source log, accurate chunk reads, and complete coverage of the claimed scope.

4 DBLog Claims and Limitations

DBLog combines source-log events with chunk-read observations. A live backfill establishes an initial virtual cut, and later CDC events can advance that cut to new source frontiers.

The result is a source-side replay claim on a named scope and frontier. It does not remove the need for source coordination or establish the state of a destination.

4.1 What DBLog Provides

The protocol combines six primary inputs:

- a claimed key scope.
- a certified frontier in source-log order.
- chunk reads whose observations become refresh events.
- ordinary CDC events from the source log.
- source-log and chunk-read evidence sufficient to justify the clean prefix.
- a finite replay prefix whose *apply* state can be compared with the source.

Replaying the clean prefix yields the exact per-key state of the source at the frontier on the claimed scope. Once this equality has been established, appending the source changes committed after that frontier and through a later frontier advances the cut. Neither step requires a physical snapshot read.

4.2 Limits of the Result

The result depends on source-log ordering, retained CDC events, chunk-read coordinates, frontier ordering, and evidence that connects these observations to the source. A deployment or source connector must establish these conditions.

The core theorem guarantees equality specifically for the claimed scope. Whole-table equality is a derived result requiring the Layer 4 anchor-domain and no-unclaimed-replay-key conditions.

The theorem does not establish destination state or exactly-once delivery. A destination may receive the certified replay without applying it in the required order or reaching the corresponding state. The source-side continuation result advances replay equality to later frontiers, but it does not show that a sink has applied those events. Section 12 states this limit.

A virtual cut also does not imply that one physical source-read timestamp existed for all chunk rows. It is an equality of outcomes.

Debezium, Apache Flink CDC, MySQL, PostgreSQL, and MariaDB provide deployment and related-work context. The theorem does not apply to any named system until its implementation and source assumptions have been checked against the formal model.

4.3 Failure Modes and Required Evidence

Table 2 lists the main failure modes, the evidence or assumption that addresses each one, and whether the condition is machine-checked or supplied by the deployment.

For the initial certificate, a checker can expose the internal evidence failures listed in the table. Layer 4 adds the conditions needed for a whole-table claim. A concrete checker for continuation certificates is not included. In all cases, the theorem assumes that the source observations match the mathematical source history.

4.4 Trust Boundary

The strongest result remains source-side. Under accurate source observation and the checker assumptions, accepted evidence yields a certified virtual cut. The result does not prove that a concrete parser is correct, that a connector reports the source accurately, that a named product implements every assumption, or that a sink has applied the replay.

5 Formal Model (Layer 0 and Layer 1)

Layer 0 defines source histories, source state at a frontier, replay events, replay application, and restriction to a key scope. Layer 1 defines abstract DBLog runs, including chunks, watermarks, chunk reads, CDC observations, clean prefixes, and wellformedness conditions.

These layers do not include certificates, verifiers, destination ACKs, or whole-table conditions. Layer 2 proves a theorem about wellformed runs. Layer 3 then establishes when an accepted certificate guarantees such a run.

Table 3 fixes the symbols the formal sections use. A predicate (short sans-serif tag like WF, FSO, Mat, Coh, VC, WTS) is a relation that holds or fails on its arguments. An accessor function (italic word like *apply*, *src*, *scope*, *frontier*, *prefix*) returns a value.

5.1 Source Histories and Frontiers

A source database is modeled as a base state b_0 together with a finite history H of committed source events. A source event is one of

Insert k v , Update k v , Delete k .

Each event records a source coordinate, such as a commit position, log sequence number, monotonic timestamp, or equivalent order-bearing token. We write $c \leq c'$ when coordinate c is no later than c' . A source history is wellformed when its events are ordered by that coordinate, the base coordinate sits before every event, and same-coordinate events have a deterministic source-position order inside the history.

The source state at a frontier is written $src(b_0, H, f)$. For each key k , this state looks for the latest source event for k at or before frontier f . An insert or update makes the key present with the written value. A delete makes it absent. If there is no event, the key keeps its base state $b_0(k)$.

Table 2: Failure modes, required evidence or assumptions, and their status.

Failure mode	What goes wrong	Evidence or assumption boundary	Status
Missing CDC event	A source write in the claimed interval is absent from the replay evidence.	Source-log retention and CDC coverage through the certified frontier.	Deployment obligation / External observation assumption
Missing continuation segment	A later frontier is claimed without the actual source-log change data the source committed between the certified frontier and that later frontier.	Source-log retention for the continuation interval and complete CDC coverage.	Deployment obligation / External observation assumption
Bad watermark or coordinate	A chunk row is treated as if it was read at the wrong source-log position.	Watermark or coordinate validity for the chunk read.	Deployment obligation / External observation assumption
Stale refresh survives	An older chunk-read value overrides a newer source-log event for the same key.	Clean-prefix dominance and latest-event ordering.	Machine-checked
Wrong frontier	The replay is certified against a frontier different from the source interval it actually covers.	Frontier consistency in the certificate and evidence.	Machine-checked
Scope ownership mismatch	A claimed key is not owned by exactly one responsible chunk, or the claimed scope does not match the chunk plan.	Chunk ownership and claim-scope checks.	Machine-checked
Incomplete chunk-plan termination	The chunk loop stops before the intended end of the key range, so a key in the intended operational scope may be absent from the finite plan. For a whole-table claim, the anchor domain is not shown to be covered.	Chunk-plan construction, maximum-key or end-of-scope evidence, primary-key ordering, and whole-table claim-scope coverage.	Deployment obligation / External observation assumption
Whole-table replay-key mismatch	The whole-table specialization is requested while replay evidence materializes keys outside the claim scope.	Layer 4 no-unclaimed-replay-key side condition, which is not part of scoped virtual-cut equality.	Machine-checked
Inaccurate observation	The connector, source log, primary-key interpretation, or chunk-read result does not accurately reflect the source database.	Not internally provable from the certificate alone.	External observation assumption

The coordinate convention is inclusive. A chunk read recorded at coordinate c observes $src(b_0, H, c)$, including source events at coordinate c . A chunk read is *attributed* to a coordinate within its watermark bracket at which all its row images match the committed source state. §3.2 sets out what a deployment must supply for that attribution to be sound. The watermark bracket locates the read, while chunk-read semantics or coordinate evidence establish that all returned row images match that common coordinate. Canonical replay therefore places same-coordinate CDC evidence before the refresh that records the post- c chunk-read state. Later source-coordinate events still appear after that refresh.

This is the source-side reference state that every virtual-cut theorem compares against.

5.2 Replay Events and Apply

DBLog does not replay source events alone. It replays two kinds of replay event:

$Cdc\ c\ e$ source-log event e at coord. c ,
 $Refresh\ k\ m\ c$ chunk-read of k at coord. c .

In a refresh, $m = \text{Some } v$ means the chunk read observed value v . The value $m = \text{None}$ means that the row was absent. Absence is part of the state. Replaying a refresh with None deletes the key from the replay state.

The replay function is $apply(\sigma)$, where σ is a finite replay sequence. $apply$ walks the sequence from left to right, updating a per-key map. Later events for the same key

Table 3: Notation used in the formal model and theorem statements. Predicates are rendered in sans-serif. Accessor functions are *italic*.

Symbol	Kind	Read as
b_0, H, f	variables	base state, source history, frontier coordinate.
b, f'	variables	whole-table anchor source coordinate ($b \leq f$) and later source frontier as the continuation target ($f \leq f'$).
R, C, E	variables	abstract DBLog run, certificate, evidence bundle (§8.2).
σ, δ, K	variables	replay sequence, continuation segment (CDC replay-event list), key scope.
$src(b_0, H, f)$	accessor	source state at frontier f .
$apply(\sigma)$	accessor	replay-apply of sequence σ .
$scope(\cdot)$	accessor	claimed key scope (of run or certificate).
$frontier(\cdot)$	accessor	chosen source-log frontier.
$prefix(\cdot)$	accessor	clean replay prefix.
$m \upharpoonright K$	operator	state m restricted to keys in K .
$verify(C, E)$	accessor	verifier result, one of <i>Accept</i> , <i>Reject</i> , <i>Unsupported</i> .
raw	variable	raw source-observation carrier (External observation assumption).
$anchorDom(H, b)$	accessor	keys present at the anchor boundary b .
$touched(H, b, f)$	accessor	keys with source events in $(b, f]$.
$tblScope(H, b, f)$	accessor	union of the two above.
$keys(\sigma)$	accessor	keys written by replay sequence σ .
$chunks(R)$	accessor	run’s chunk plan.
$srcHistory(R)$	accessor	source history the run is about.
$cdcEvents(R)$	accessor	CDC events the run consumed.
$WF(b_0, R, H)$	predicate	wellformed DBLog run (Layer 1).
$VCS(\sigma, K, f, H)$	predicate	state-level virtual cut (Section 6). The base state b_0 is implicit in this four-argument shorthand and made explicit in the unfolded form in Section 6.
$VC(C, H)$	predicate	certificate-side virtual cut.
$ContSeg(H, K, f, f', \delta)$	predicate	δ is the continuation segment for $(f, f']$ on K (Section 11).
$Mat(C, E, R)$	predicate	(C, E) materializes abstract run R .
$Coh(C, E, R)$	predicate	(C, E) is coherent with run R (Section 8.4).
$FSO(E, b_0, H, raw)$	assumption	accurate source observation (FSO).
$WTS(C, H, b)$	predicate	whole-table claim scope (Layer 4).

win. *apply* starts from the all-absent per-key map. Formally, $apply([])(k) = \text{None}$ for every key k . Thus the running example’s later update for Bob overrides chunk A’s older refresh, and a later delete would override both an earlier refresh and an earlier update.

State restriction is written $m \upharpoonright K$ in prose formulas below. It keeps only the entries whose keys lie in K , so $(m \upharpoonright K)(k) = m(k)$ when $k \in K$, and *None* otherwise. The displayed formulas use the $\upharpoonright$ symbol throughout.

5.3 Abstract DBLog Runs

A DBLog run R is modeled abstractly, characterized through six accessor functions:

- $scope(R)$ claimed key scope,
- $frontier(R)$ chosen source frontier,
- $prefix(R)$ replay sequence of the run,
- $chunks(R)$ run’s chunk plan,
- $srcHistory(R)$ source history the run is about,
- $cdcEvents(R)$ observed CDC events consumed.

Each chunk has a domain of keys, a responsible-chunk relation from keys to chunks, a lower and upper watermark, and a chunk-read coordinate. The model uses a strict canonical partition. Each key in $scope(R)$ belongs to exactly one responsible chunk, and every responsible chunk has a finite domain. The model also takes the chunk plan itself to be finite, so every run has a finite set of chunks. This models a backfill with a bounded chunk loop over a finite plan.

That strict partition is not a claim that real systems never retry or overlap chunk reads. It is the canonical run selected for the theorem. Raw overlap/retry evidence must be normalized into such a canonical run, or handled by a future extension.

5.4 Clean Prefix

The clean prefix of a run is the finite replay sequence constructed from three elements:

- chunk-read refreshes for the responsible chunks.
- observed CDC events for in-scope keys through the frontier.
- the *clean-prefix ordering*, sorting events by source coordinate, placing CDC events before chunk refreshes at shared

Table 4: Wellformed-DBLog-run obligations in $WF(b_0, R, H)$ and their status.

Obligation	Meaning	Status
Source binding	R is about source history H , and H is wellformed.	Deployment obligation
Chunk partition	The chunks form a finite strict partition of $scope(R)$.	Deployment obligation
CDC coverage	Every relevant source event after the responsible chunk read and through the frontier is observed.	External observation assumption
CDC fidelity	Observed CDC events are real source events, in the right order, with same-key same-coordinate multiplicity preserved.	External observation assumption
Chunk evidence	Chunk read coordinates are bracketed by watermarks, and chunk-read observations produce the corresponding value-or-absence refreshes for claimed keys.	Deployment obligation
Read-before-frontier	Every chunk read occurs no later than the run frontier.	Deployment obligation
Refresh fidelity	A chunk-read value or absence matches $src(b_0, H, \text{read-coord.})$ for that key.	External observation assumption / Deployment obligation

coordinates, and keeping same-coordinate CDC events in source-log position order.

The clean prefix keeps every chunk refresh. The optional normalized presentation drops dominated refreshes without changing the final replay state.

The theorem does not rely on an unstated “good replay” primitive. The clean prefix is constrained by the run’s chunk plan, CDC observations, frontier, and scope. If a needed CDC event is missing, if a future event is replayed, if a refresh appears without a responsible chunk observation, or if same-coordinate CDC events are reordered, the run is not wellformed for the theorem.

The main invariant is per-key. For each key in scope, the latest replay event in the clean prefix is either the latest accurate source event through the frontier, or it is a refresh whose value or absence at its read coordinate remains unchanged until the frontier. Layer 2 derives its result from this invariant.

The mechanization has one additional assumption. The canonical clean prefix enumerates each chunk domain in sorted key order, so the run- and certificate-layer theorems assume a linear order on the key type. This introduces no additional requirement, as chunk selection already assumes linearly ordered primary keys. The state-level continuation and restriction theorems of Section 11 carry no such constraint.

5.5 Wellformed Runs

We write $WF(b_0, R, H)$ for the wellformed DBLog run predicate. It collects the source-side conditions used by the proof. Table 4 lists these conditions.

The split between Deployment obligations and External observation assumptions is intentional. The model defines what a wellformed run must satisfy. It cannot prove, from within the run model itself, that an external database connector reported every source event without loss or that a chunk read accurately captured the real source state.

Refresh fidelity includes both labels because it combines two aspects. A returned row image must report the source

value or absence accurately, which is an External observation assumption. The chunk-read semantics pin those images to a single coordinate (§3.2), which is a Deployment obligation.

The predicate has one further conjunct, clean-prefix closure, omitted from the table because it follows from the construction in Section 5.4 rather than from a deployment or external observation. The proof may therefore use it without adding a deployment obligation.

The lower and upper watermarks introduced above are source-log coordinates, not primary-key-range bounds. Accordingly, the Chunk partition row is a statement about the finite canonical partition consumed by the theorem. It is not a proof that a concrete chunk-selection loop stopped in the right place, and Layer 1 contains no maximum-key accessor or chunk-loop termination algorithm. A maximum-primary-key query, an empty-next-chunk observation, a predeclared range bound, or another end-of-scope indicator may justify the finite partition operationally. Supplying the resulting partition, including evidence that the loop reached the intended end of the claimed key scope, is a Deployment obligation. The theorem consumes the partition, not the plan-construction algorithm.

5.6 Layer 0 and Layer 1 Lemmas

The source and replay model establishes the elementary facts used by the main proof:

- *apply* is latest-event-wins per key.
- Restricted states are equal exactly when they agree on every key in the scope.
- *src* is characterized by the latest source event at or before the frontier.
- A replay-side Cdc event has the same per-key effect as its source event.
- A refresh has the source-state value at its chunk-read coordinate when the chunk-read fidelity assumption holds.
- The source state at a later frontier is the source state at an earlier frontier updated by exactly the source events strictly between them. This is immediate from the preceding

characterization, since the source events at or before f' are those at or before f together with those in $(f, f']$. This interval characterization is what the continuation result of Section 11 builds on.

The run layer then contributes the bridge lemma, displayed here as a preview of the Layer 2 theorem stated and explained in Section 7:

$$\begin{aligned} \text{WF}(b_0, R, H) &\implies \text{apply}(\text{prefix}(R)) \upharpoonright \text{scope}(R) \\ &= \text{src}(b_0, H, \text{frontier}(R)) \upharpoonright \text{scope}(R). \end{aligned}$$

Section 6 first names the equality itself.

A wellformed run is bound to its source history. It requires a finite chunk partition, watermark-bracketed chunk reads, and complete, accurate CDC evidence. The following theorems apply to runs that meet these conditions.

6 Virtual Cuts

A virtual cut formalizes the property that replaying a prefix yields the same per-key state as the source at a designated frontier on a given scope. It establishes an equality of outcomes rather than requiring a simultaneous physical read cut.

We write $\text{VCS}(\sigma, K, f, H)$ for the state-level form, with the base state b_0 left implicit in this four-argument shorthand:

$$\text{VCS}(\sigma, K, f, H) \iff \text{apply}(\sigma) \upharpoonright K = \text{src}(b_0, H, f) \upharpoonright K.$$

Here σ is a replay sequence, K is a key scope, f is a frontier, and H is the source history. The base state b_0 is fixed by context. When needed, the formulas write it explicitly.

The certificate-side form $\text{VC}(C, H)$ specializes the same equality to the accessors of a certificate:

$$\text{VC}(C, H) \iff \text{VCS}(\text{prefix}(C), \text{scope}(C), \text{frontier}(C), H).$$

Expanding this definition:

$$\begin{aligned} \text{apply}(\text{prefix}(C)) \upharpoonright \text{scope}(C) \\ = \text{src}(b_0, H, \text{frontier}(C)) \upharpoonright \text{scope}(C). \end{aligned}$$

The definition is extensional. It says that two per-key states are equal on a scope at a frontier. It does not say that every chunk row was read at that frontier, that one physical source snapshot existed, that the scope is the whole table, or that any destination has applied the replay. Figure 4 contrasts a physical cut with the virtual cut this paper proves.

This is the exact shape illustrated by the running example. There, the replay sequence has seven items, the scope is $\{100, 200, 300\}$, the frontier is c_4 , and apply produces the same balances as src at c_4 on those three keys. Chunk A and chunk B were read at different coordinates. The virtual cut does not hide that fact. It says the replay outcome is snapshot-equivalent on the claimed keys.

The state-level form is useful because Layer 2 proves the equality for run accessors:

$$\text{VCS}(\text{prefix}(R), \text{scope}(R), \text{frontier}(R), H).$$

Layer 3 then proves that an accepted certificate, under accurate source observation, certifies a run whose accessors agree with the certificate. That is how the run-side theorem becomes the certificate-side theorem.

7 Source-Side Virtual-Cut Soundness (Layer 2)

Layer 2 proves the source-side theorem:

$$\begin{aligned} \text{WF}(b_0, R, H) &\implies \text{apply}(\text{prefix}(R)) \upharpoonright \text{scope}(R) \\ &= \text{src}(b_0, H, \text{frontier}(R)) \upharpoonright \text{scope}(R). \end{aligned}$$

Theorem Summary.

- **Status.** Machine-checked under the displayed assumptions.
- **Claim.** The clean prefix of a wellformed DBLog run reaches the source state at the run frontier on the run scope.
- **Assumptions.** The wellformed-run obligations in Section 5.5, including chunk partition, CDC coverage and fidelity, chunk-read fidelity, and read-before-frontier ordering.
- **Boundary.** This is a source-side run theorem, distinct from certificate-checking, whole-table, or sink-state theorems.

Equivalently, every wellformed DBLog run satisfies the virtual-cut-state equality at the run's frontier on the run's scope. The theorem is stated about the run's unnormalized replay prefix. By the dominated-refresh equivalence of §3.4, the normalized clean prefix yields the same apply state and therefore the same equality.

The theorem assumes the wellformed-run predicate of Section 5.5, including chunk partition, frontier bounds, CDC coverage and fidelity, and chunk-read fidelity. Each condition is stated explicitly because omitting it allows a counterexample (Table 5). The predicate's remaining conjunct, clean-prefix closure, is derived by definition. It holds for every run by the clean-prefix construction, so the proof may use it but no deployment need establish it separately.

7.1 Proof Idea

The proof is a per-key latest-event argument.

First, restricted-state equality reduces the theorem to point-wise equality. It is enough to show that for every key $k \in \text{scope}(R)$,

$$\text{apply}(\text{prefix}(R))(k) = \text{src}(b_0, H, \text{frontier}(R))(k).$$

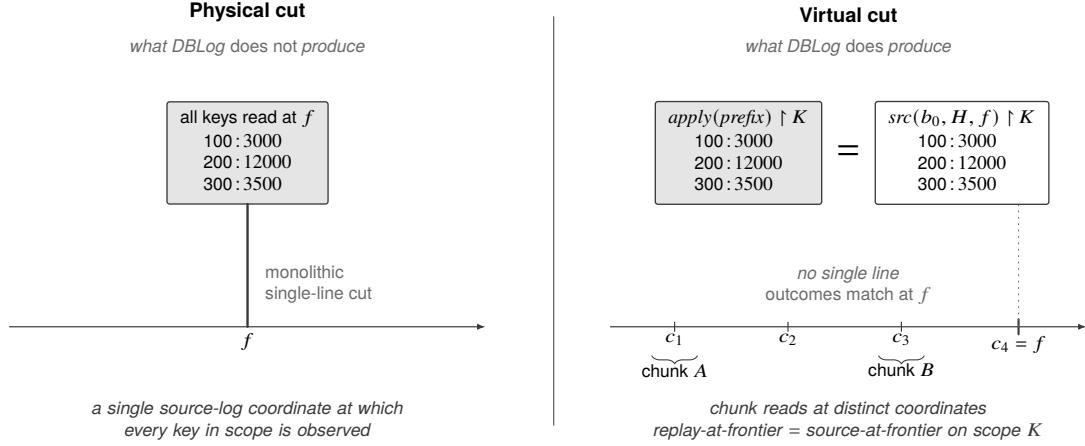


Figure 4: Physical cut vs. virtual cut. *Left*: a physical cut requires a single source-log coordinate at which every key in scope is observed, shown as a single vertical line. DBLog does *not* produce one. The values shown are the counterfactual outcome on K . *Right*: a virtual cut requires replay at the frontier to equal the source at the frontier on the chosen scope. The chunk reads (as in Figure 2) sit at distinct earlier coordinates, but the replay reconstructs the source state at the frontier.

Fix such a key k . Since the run is wellformed, k has a unique responsible chunk, and the chunk-read evidence produces a value-or-absence refresh for k that belongs to $prefix(R)$, including Refresh k None c when the row is absent at the chunk-read coordinate. Therefore the “no replay event for k ” case cannot occur for an in-scope key. By the clean-prefix construction, every in-scope key k has a latest event in $prefix(R)$, which must be either a CDC event or the responsible chunk’s refresh. The proof therefore has two cases.

7.2 Case 1. The Latest Replay Event Is CDC

Suppose the latest replay event for k in $prefix(R)$ is a CDC event $Cdc\ c\ e$. Clean-prefix coherence ensures this replay event originates from an observed CDC pair, while CDC fidelity guarantees that this pair corresponds to a genuine source event in H . Because the event lies in the clean prefix, its source coordinate c is within the run’s frontier, i.e. $c \leq frontier(R)$. Order preservation and same-coordinate multiplicity ensure that the replay preserves both the order and the occurrences of source events for that key and coordinate.

Now assume, for contradiction, that the source history has a later event for k at or before the frontier. CDC coverage would put that later event into the observed CDC list. Clean-prefix closure would put it into the replay. The clean-prefix ordering would place it after $Cdc\ c\ e$, contradicting the choice of $Cdc\ c\ e$ as the latest replay event for k .

So the latest replay CDC event corresponds to the latest source event for k through the frontier. $apply$ and src therefore make the same update at k .

7.3 Case 2. The Latest Replay Event Is a Refresh

Suppose the latest replay event for k is Refresh $k\ m\ c_{read}$. The responsible-chunk obligations identify the responsible chunk, and refresh fidelity ensures that

$$m = src(b_0, H, c_{read})(k).$$

If there were a later source event for k in $(c_{read}, frontier(R)]$, CDC coverage and clean-prefix closure would place a corresponding Cdc event after the refresh in the clean prefix. The interval is open at c_{read} because a read at c_{read} already observes source events at c_{read} . The post-read repair obligation starts after the read coordinate and runs through the frontier. A later source event would contradict the refresh being latest for k .

Therefore the source state for k at the read coordinate is still the source state for k at the frontier. The refresh writes exactly that value or absence, so $apply(prefix(R))(k)$ equals $src(b_0, H, frontier(R))(k)$. Both row presence and row absence satisfy this equation, with absence represented by None in the per-key state.

7.4 Why the Assumptions Are Necessary

The proof fails if the obligations are weakened.

These negative cases are the reason the theorem requires the full wellformed-run predicate rather than a vague statement that DBLog “collected some chunks and some CDC.”

A wellformed backfill produces a replay sequence that reconstructs the source state at the chosen frontier on the chosen scope, requiring neither long-lived table locks nor pauses in source writes. The next two layers connect this

Table 5: Counterexamples obtained when selected obligations in WF are removed.

Missing obligation	Counterexample shape	Boundary protected
CDC coverage	A source update after a chunk read is missing from the replay, allowing a stale refresh value to persist.	DBLog needs retained source-log evidence.
Strict chunk ownership	Two chunks both claim a key, or no chunk owns it, so the refresh source is ambiguous or absent.	The theorem is scoped through a canonical chunk plan.
Read-before-frontier	A chunk read after the frontier injects a future value into the replay.	The replay is frontier-bound.
Same-coordinate order and multiplicity	Duplicate or same-coordinate source events are observed in a different order or with an occurrence missing.	Latest-event reasoning must match source order.
Refresh fidelity	A chunk refresh records a value or absence that was not the source state at the read coordinate.	Chunk reads remain an External observation assumption.
Clean-prefix order	A clean-prefix event for some key sits before an earlier-coordinate same-key event, so the prefix’s latest event for that key differs from the source’s.	Per-key latest-event reasoning transfers from the prefix to the source.

run-level guarantee to an inspectable certificate and, under additional conditions, extend it to whole-table equivalence.

8 Certificates, Evidence, and Verifiers

Layer 2 proves a theorem about an abstract DBLog run. If the run is wellformed, its clean replay prefix reaches the source state at its frontier on its scope. In practice, a deployment provides a certificate and associated evidence bundle rather than an abstract run. This section defines those artifacts, and Section 9 gives the corresponding theorem.

8.1 Certificate

A **certificate** is the claim object, specifying three accessors:

- $scope(C)$ the keys being claimed,
- $frontier(C)$ the source-log coordinate of the claim,
- $prefix(C)$ the certified replay prefix.

Before acceptance, $prefix(C)$ represents only a claimed prefix. Storing a replay sequence in a certificate provides no guarantee of validity. The certificate theorem applies only after accepted evidence materializes a coherent, wellformed run that agrees with the certificate’s accessors.

8.2 Evidence

An **evidence bundle** contains the source-side material the checker inspects. This includes certificate binding evidence, the chunk plan, chunk-read coordinates, chunk domains, observed row values or absences, ordinary CDC observations, and the construction evidence for the raw clean prefix.

The evidence bundle does not contain a destination-state proof. It does not contain sink ACK semantics. It does not contain whole-table claim-scope side conditions except where

those are added later for the whole-table specialization. The certificate layer is source-side.

8.3 Materialized Run

The materialization relation $\text{Mat}(C, E, R)$ connects certificate C and evidence E to the model. Together, they describe the abstract DBLog run R consumed by the Layer 2 theorem. Materialization is purely structural: it connects the evidence to an abstract run carrier (specifying scope, frontier, clean prefix, chunk plan, chunk reads, and CDC events) without asserting that the resulting run is wellformed against the source history.

Concretely, Mat relates the pair to the evidence-determined run. The materialized run’s scope, frontier, clean prefix, chunk plan, chunk-read observations, and CDC observations are fixed by what E serializes and cannot be chosen arbitrarily by the checker. Evidence that does not describe a run shape the checker supports is reported **Unsupported** instead of being accepted into the claim. The certificate’s own fields are tied to the materialized run by the coherence predicate below. Materialization alone establishes neither wellformedness nor observation fidelity. Those properties derive from the coherence predicate and the checker obligations.

Rejected and unsupported evidence need not materialize a run. For accepted evidence, the coherence predicate ensures that the checker uses the run determined by the evidence.

8.4 Certificate / Run Coherence

We write $\text{Coh}(C, E, R)$ for the certificate / run coherence predicate. It packages materialization with accessor agreement between C and R :

Table 6: Verifier result cases.

Result	Meaning
Accept	The certificate/evidence pair is internally coherent for the source-side checker fragment.
Reject	The evidence contradicts the certificate or fails a required check.
Unsupported	The evidence uses source-side vocabulary outside the checker fragment modeled here.

$$\begin{aligned}
\text{Coh}(C, E, R) &\iff \text{Mat}(C, E, R) \\
&\quad \wedge \text{scope}(C) = \text{scope}(R) \\
&\quad \wedge \text{frontier}(C) = \text{frontier}(R) \\
&\quad \wedge \text{prefix}(C) = \text{prefix}(R).
\end{aligned}$$

Coherence concerns the run materialized by this certificate and evidence pair. It is stronger than the existence of some run with matching accessors.

8.5 Verifier Result

The verifier produces one of three outcomes (Table 6):

The **Unsupported** outcome prevents unsound acceptance when the verifier encounters evidence outside its modeled fragment. If the checker cannot interpret an optional source-side feature, the certificate is not accepted as if that feature had been checked.

Accept, **Reject**, and **Unsupported** are verifier results, not theorem-status labels. The verifier result here is an abstract source-side checker result, not a claim that a serialized certificate format, parser, or production verifier implementation has been proved correct.

8.6 What Accept Means, and What It Does Not Mean

Accept indicates that, under accurate source observation, the evidence materializes a wellformed DBLog run whose scope, frontier, and clean prefix match the certificate. That provides the bridge back to Layer 2.

Accept does not validate the concrete database, connector, log, primary-key interpretation, source-coordinate semantics, parser, wire format, or named product. It also does not show that a sink applied the prefix or that the claim covers the whole table. Accurate source observation remains an explicit assumption.

8.7 Accurate Source Observation

We write $\text{FSO}(E, b_0, H, \text{raw})$ for the accurate source observation assumption (FSO), the External observation assumption of the certificate layer. It ties the finite evidence bundle E to

the mathematical source history H and base state b_0 , against an abstract observation carrier raw . It covers the following source facts:

- the source history and base state the certificate is about.
- the meaning and ordering of source-log coordinates.
- CDC retention and completeness over the certified interval.
- preservation of same-coordinate order and multiplicity where relevant.
- accuracy of chunk-read values and absences at their recorded coordinates.
- the primary-key interpretation and the assumption that chunk reads observe committed source state.

The theorem statements write the raw source-observation carrier as raw . The carrier is abstract. It stands for a deployment’s record of source observations, interpreted under the accurate-source-observation assumption (FSO). It is not a destination log, not a sink-state proof, and not a claim that a named connector has been verified.

The certificate checker can compare evidence fields to one another. It cannot prove, from the certificate alone, that the outside world produced accurate observations. Consequently, the accepted-certificate theorem takes accurate source observation as an explicit External observation assumption.

8.8 Role of the Certificate Layer

Layer 2 needs a wellformed mathematical run, while a deployment exposes a certificate and evidence. Layer 3 connects these levels by establishing that accepted evidence materializes a coherent wellformed run under accurate source observation, enabling the application of the source-side virtual-cut theorem.

9 Accepted-Certificate Soundness (Layer 3)

Layer 3 establishes the certificate-side theorem across two components: an intermediate bridge theorem and the composed virtual-cut theorem.

9.1 Intermediate Theorem

The intermediate theorem is machine-checked under a small set of primitive *checker obligations* that the abstract certificate checker must meet to guarantee a valid run. Table 7 lists them. They are stated as explicit assumptions. We do not prove that a concrete parser, wire format, or production verifier discharges them.

Obligations S1–S4 form the assumptions of the intermediate theorem. Each is a property that the checker design must satisfy before an **Accept** result implies the theorem’s source-side conclusion. Reporting unsupported evidence is not part

Table 7: Checker obligations S1–S4 for the accepted-certificate intermediate theorem.

Checker obligation	
S1	Accepted inputs are wellformed checker inputs that are structurally valid and parseable.
S2	Accepted checker inputs materialize an abstract DBLog run.
S3	The materialized run’s accessors (scope, frontier, and clean prefix) agree with the certificate.
S4	Under accurate source observation, an accepted materialization is a wellformed run against the source history.

of these formal assumptions, but rather an operational checker obligation.

Unsupported reporting. A checker meeting S1–S4 must not accept uninterpretable evidence as if it had been verified. Such evidence is reported as **Unsupported** rather than **Accept**. **Status:** Deployment obligation on the concrete checker, not a Layer 3 assumption.

Under obligations S1–S4, the intermediate theorem states:

$$\begin{aligned} \text{verify}(C, E) &= \text{Accept} \wedge \text{FSO}(E, b_0, H, \text{raw}) \\ &\implies \exists R. \text{WF}(b_0, R, H) \wedge \text{Coh}(C, E, R). \end{aligned}$$

In words, accepted evidence, when interpreted under accurate source observation, certifies a wellformed abstract DBLog run whose accessors agree with the certificate. Obligations S1–S3 guarantee materialization and accessor agreement, while S4 ensures wellformedness. The unsupported-reporting requirement prevents uninterpretable evidence from being treated as verified. This is an operational obligation rather than a formal assumption.

9.2 Composed Theorem

The main certificate theorem is:

$$\begin{aligned} \text{verify}(C, E) &= \text{Accept} \wedge \text{FSO}(E, b_0, H, \text{raw}) \\ &\implies \text{VC}(C, H). \end{aligned}$$

Expanding this definition:

$$\begin{aligned} \text{verify}(C, E) &= \text{Accept} \wedge \text{FSO}(E, b_0, H, \text{raw}) \\ &\implies \text{apply}(\text{prefix}(C)) \upharpoonright \text{scope}(C) \\ &= \text{src}(b_0, H, \text{frontier}(C)) \upharpoonright \text{scope}(C). \end{aligned}$$

Theorem Summary.

- **Status.** Machine-checked under checker obligations S1–S4 (Table 7) and accurate source observation (FSO).
- **Claim.** An accepted certificate guarantees a virtual cut when its evidence is interpreted through accurate source observation.

- **Assumptions.** Checker obligations S1 (input wellformedness), S2 (materialization), S3 (accessor agreement), S4 (accurate materialization is wellformed), and accurate source observation (FSO).
- **Boundary.** Acceptance does not prove a concrete parser, wire format, production verifier, named product, whole-table claim, or sink application. The unsupported-reporting requirement is a separate checker-design obligation rather than a formal assumption.

Consequently, certificate acceptance guarantees a virtual cut only relative to accurate source observation. Acceptance alone cannot validate physical database behavior or external connector fidelity.

9.3 Proof Explanation

The proof composes the layers already introduced.

First, verifier acceptance under accurate source observation establishes a valid run R :

$$\begin{aligned} &\text{WF}(b_0, R, H), \\ &\text{Coh}(C, E, R). \end{aligned}$$

Second, Layer 2 applies to that wellformed run:

$$\begin{aligned} &\text{apply}(\text{prefix}(R)) \upharpoonright \text{scope}(R) \\ &= \text{src}(b_0, H, \text{frontier}(R)) \upharpoonright \text{scope}(R). \end{aligned}$$

Third, coherence substitutes certificate accessors for run accessors:

$$\begin{aligned} \text{scope}(C) &= \text{scope}(R), \\ \text{frontier}(C) &= \text{frontier}(R), \\ \text{prefix}(C) &= \text{prefix}(R). \end{aligned}$$

After substitution, the Layer 2 equality becomes:

$$\begin{aligned} &\text{apply}(\text{prefix}(C)) \upharpoonright \text{scope}(C) \\ &= \text{src}(b_0, H, \text{frontier}(C)) \upharpoonright \text{scope}(C). \end{aligned}$$

Fourth, by definition of $\text{VC}(C, H)$ (Section 6), this yields the certified virtual cut.

The proof uses no sink, whole-table, or named-product assumption. Its conclusion is source-side and restricted to $\text{scope}(C)$. The checker obligations and accurate source observation remain assumptions.

9.4 Non-Vacuity and Boundary Cases

The formal development includes the following non-vacuity checks and boundary cases:

- A non-trivial accepted certificate exists with a non-base frontier, a chunk read, a value-changing replay event, and valid raw , b_0 , and H evidence.
- candidate shapes lacking required proof components violate the checker obligations.

- mismatched certificate-evidence pairs are rejected.
- unknown source-side evidence is `Unsupported` rather than silently accepted.
- mismatched certificate and run accessors fail coherence.
- inaccurate *raw*, mismatched b_0 , or corrupted H fail accurate source observation.
- an empty scope satisfies the predicate vacuously, whereas the formal development provides a non-trivial concrete instance with non-empty keys.

These cases protect the main trust boundary. The theorem does not say that the certificate eliminates trust. It states that, under accurate source observation and explicit checker obligations, acceptance establishes a coherent wellformed run. The source-side theorem then yields the certified virtual cut.

10 Whole-Table Specialization (Layer 4)

Layers 2 and 3 prove equality on $scope(C)$. In a full backfill, this scope is typically intended to cover an entire table or partition. Such a claim needs additional evidence that the scope includes every key whose state can matter between an anchor boundary and the certified frontier. The replay must also avoid writing keys outside that scope.

Whole-table equality is therefore a derived result rather than an inherent guarantee of certificate acceptance. Acceptance yields a scoped virtual cut, and the whole-table side condition establishes when the restriction to $scope(C)$ can safely be removed.

10.1 Anchor Boundary and Anchor Domain

An anchor boundary b is a source coordinate used to define the key population that the whole-table claim covers. It is not a physical snapshot read. Chunks may be read after b . Stale chunk observations can be repaired by source-log evidence and replayed to the frontier.

For a certificate C , the anchor must not be later than the certified frontier:

$$b \leq frontier(C).$$

The anchor domain is the set of keys present in the source at that boundary:

$$anchorDom(H, b) = \{ k \mid src(b_0, H, b)(k) \text{ is present} \}.$$

The anchor domain inherits its dependence on the base state b_0 from src . As with src , b_0 is a single base state fixed throughout the paper. We keep it implicit in $anchorDom(H, b)$, and likewise in the whole-table claim scope $WTS(C, H, b)$ below, since it never varies. A key present at the anchor that no source event at or before b ever writes inherits its presence

entirely from b_0 , the only place where the base state directly affects the whole-table claim.

Operationally, a whole-table deployment may use a maximum-primary-key query or equivalent end-of-table indicator to show that the claimed scope covers this anchor domain. Such an indicator does not replace WTS. It must be tied to the anchor boundary b , or otherwise justify $anchorDom(H, b) \subseteq scope(C)$. It also does not discharge the post-anchor touch condition below, including for keys beyond an anchor-time maximum.

Keys absent at b can still become relevant later. If the source inserts, updates, or deletes a key after b and at or before the certified frontier, the whole-table claim must account for that key too.

$touched(H, b, f)$ is the set of keys k for which H has an insert, update, or delete at some source coordinate c with $b < c \leq f$, and the conservative interval table scope is

$$\begin{aligned} tblScope(H, b, f) \\ = anchorDom(H, b) \cup touched(H, b, f). \end{aligned}$$

This event-touch scope is conservative rather than a minimal semantic-difference set: even deleting an already absent key counts as a touch that the claim scope must include.

10.2 A Small Anchor Example

Suppose the source contains keys 100 and 200 at anchor boundary b . Between b and frontier f , key 300 is inserted and key 200 is deleted.

$$\begin{aligned} anchorDom(H, b) &= \{100, 200\}, \\ touched(H, b, f) &= \{200, 300\}, \\ tblScope(H, b, f) &= \{100, 200, 300\}. \end{aligned}$$

A whole-table claim for this interval must include all three keys in $scope(C)$. Key 100 is present at the anchor. Key 200 is present at the anchor and then deleted. Key 300 is absent at the anchor but inserted before the frontier. Omitting any of them would make the whole-table statement stronger than the evidence supports.

There is also a replay-side condition. If the certified clean prefix writes a key outside $scope(C)$, such as key 999, scoped equality may still hold, but the no-unclaimed-replay-key condition fails. The whole-table theorem therefore does not establish unrestricted equality for that prefix.

10.3 Whole-Table Claim Scope

We write $WTS(C, H, b)$ for the whole-table claim-scope side condition, defined by four conjuncts:

$$\begin{aligned}
& \text{WTS}(C, H, b) \iff b \leq \text{frontier}(C) \\
& \wedge \text{anchorDom}(H, b) \subseteq \text{scope}(C) \\
& \wedge \text{touched}(H, b, \text{frontier}(C)) \subseteq \text{scope}(C) \\
& \wedge \text{keys}(\text{prefix}(C)) \subseteq \text{scope}(C).
\end{aligned}$$

The first conjunct fixes the ordering of the anchor and frontier. The next two require $\text{scope}(C)$ to cover every key present at the anchor and every key touched between the anchor and the frontier. The final conjunct ensures that the certified replay prefix writes no unclaimed keys. Figure 5 illustrates the four conjuncts as overlapping key regions (anchor-domain, touched-between, replay-keys) over the source-coordinate axis.

This predicate is explicit because whole-table correctness is not automatic. Certificate acceptance alone gives a scoped statement. The whole-table side condition is the extra bridge from “correct on the certified claim scope” to “correct on every table key”.

10.4 Whole-Table Theorem

Under certificate acceptance, accurate source observation, and the whole-table claim-scope side condition, the certified clean prefix reaches the source state at the certified frontier on the whole table:

$$\begin{aligned}
& \text{verify}(C, E) = \text{Accept} \wedge \text{FSO}(E, b_0, H, \text{raw}) \\
& \wedge \text{WTS}(C, H, b) \\
& \implies \text{apply}(\text{prefix}(C)) = \text{src}(b_0, H, \text{frontier}(C)).
\end{aligned}$$

Theorem Summary.

- **Status.** Machine-checked under checker obligations S1–S4 (Table 7), accurate source observation (FSO), and $\text{WTS}(C, H, b)$.
- **Claim.** The scoped certificate theorem lifts to whole-table source-state equality when the whole-table claim-scope side conditions cover boundary order, anchor-domain keys, post-anchor touches, and replay keys.
- **Assumptions.** Certificate acceptance, accurate source observation, $\text{WTS}(C, H, b)$, and checker obligations S1–S4 (Table 7).
- **Boundary.** This remains a source-side theorem, distinct from sink convergence, exactly-once delivery, or arbitrary destination correctness.

This is a source-side theorem. The certificate checker and accurate-source-observation assumption supply the certified scoped virtual cut. The whole-table side condition supplies the coverage needed to remove the scope restriction.

10.5 Proof Outline

Layer 3 gives the scoped equality:

$$\begin{aligned}
& \text{apply}(\text{prefix}(C)) \upharpoonright \text{scope}(C) \\
& = \text{src}(b_0, H, \text{frontier}(C)) \upharpoonright \text{scope}(C).
\end{aligned}$$

For keys inside $\text{scope}(C)$, that is already the desired equality. For keys outside $\text{scope}(C)$, the whole-table side condition addresses both source and replay states via the inclusions:

$$\begin{aligned}
& \text{anchorDom}(H, b) \subseteq \text{scope}(C), \\
& \text{touched}(H, b, \text{frontier}(C)) \subseteq \text{scope}(C), \\
& \text{keys}(\text{prefix}(C)) \subseteq \text{scope}(C)
\end{aligned}$$

By contraposition, any key $k \notin \text{scope}(C)$ belongs neither to $\text{anchorDom}(H, b)$, nor to $\text{touched}(H, b, \text{frontier}(C))$, nor to $\text{keys}(\text{prefix}(C))$. First, because $k \notin \text{anchorDom}(H, b)$, it is absent at the anchor boundary (even with base state b_0 taken into account). Because $b \leq \text{frontier}(C)$ and k is not touched in $(b, \text{frontier}(C)]$, k remains absent from the source at the frontier. Second, because $k \notin \text{keys}(\text{prefix}(C))$, the clean prefix never writes k . Replay and source are therefore both absent outside $\text{scope}(C)$. Combining the inside-scope and outside-scope cases gives unrestricted equality.

10.6 Scope of the Result

The specialization does not change the paper’s trust boundaries, continuing to rely on accurate observation of source-side evidence. It does not prove that a concrete parser or production verifier is correct, nor does it establish sink-state convergence. It also distinguishes a virtual cut from a physical cut. The anchor provides a source-coordinate boundary for reasoning about table scope rather than representing a single request-wide read timestamp.

11 Source-Side Continuation and Restriction of Virtual Cuts

The results so far fix replay equality at one frontier. After the chunk loop closes, DBLog continues consuming source-log events. We now show how the established equality advances with that stream.

Continuation across frontiers appends the source changes committed after frontier f and through a later frontier f' . Restriction shows that a virtual cut on scope K remains valid on every smaller scope. Both results are source-side and machine-checked. They contain no destination, replica, or delivery term. Disjoint composition, verification of extended certificates, and normalization of overlapping or retried chunks are not proved here.

Both proofs proceed per key. Continuation uses the Layer 0 fact that the source state at f' is determined by the state at f together with the events in $(f, f']$. Restriction follows directly from equality on each key in the original scope.

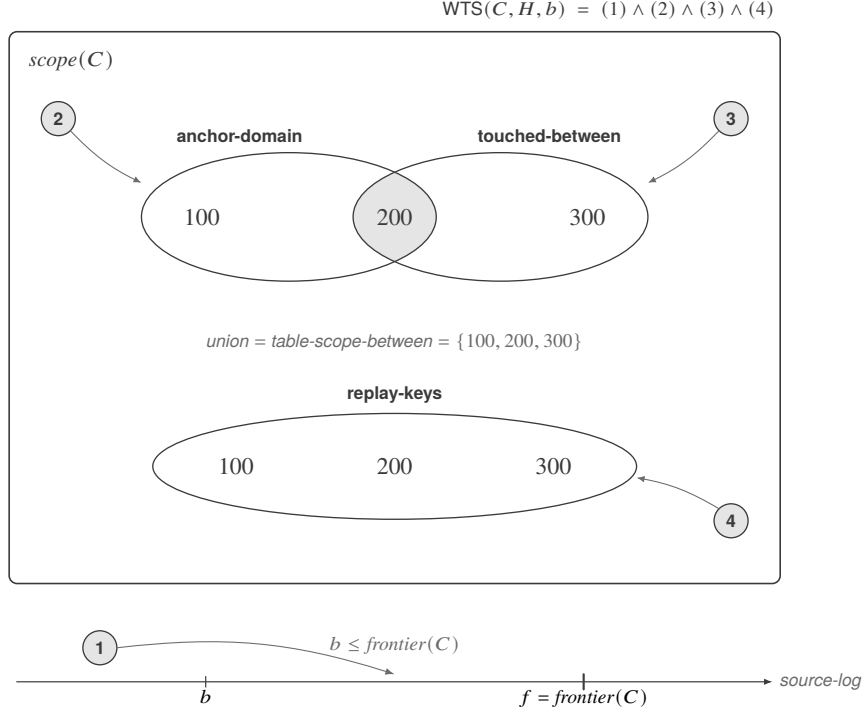


Figure 5: Anchor domain and whole-table claim scope. The four numbered conjuncts of $WTS(C, H, b)$ are annotated on the diagram. The base state b_0 , fixed by context, is implicit in WTS and $anchorDom$. (1) The anchor b precedes the frontier on the source-log axis: $b \leq frontier(C)$. (2) $anchorDom(H, b) \subseteq scope(C)$. (3) $touched(H, b, frontier(C)) \subseteq scope(C)$. (4) $keys(prefix(C)) \subseteq scope(C)$. Concrete keys follow the worked example of Section 10.2: 100 sits in anchor-domain only, 200 in the overlap, 300 in touched-between only. Replay-keys are $\{100, 200, 300\}$.

11.1 Continuation Segments

For a source history H , a key scope K , and source frontiers $f \leq f'$, the continuation segment for the half-open interval $(f, f']$ on K is obtained by filtering H . We keep every source event whose coordinate c satisfies $f < c \leq f'$ and whose key lies in K . The events remain in source order and are read as CDC replay events. We write $ContSeg(H, K, f, f', \delta)$ when δ is exactly this list, with source order and multiplicity preserved.

By construction, $ContSeg(H, K, f, f', \delta)$ requires $f \leq f'$. The predicate is false whenever $f > f'$, independent of δ . Theorems whose assumptions include $ContSeg(H, K, f, f', \delta)$ therefore implicitly require $f \leq f'$. The order is not restated in their displayed formulas.

The interval is half-open because coordinate indexing is inclusive (Section 5.1). Because $src(b_0, H, f)$ already incorporates all events through coordinate f , continuation begins strictly after f and extends through f' inclusive.

Because the segment is defined by filtering the source history, a list that reorders, drops, or duplicates a source event does not satisfy $ContSeg(H, K, f, f', \delta)$. A production CDC feed must deliver this exact sequence. Its accuracy is an **External observation assumption**, and retaining the

source-log interval $(f, f']$ is a **Deployment obligation**.

The segment is filtered to the scope K . It is the change data for the keys of K , not the source's entire change stream. A deployment whose CDC feed includes all keys must project that stream onto K to obtain the continuation segment for a K -scoped cut. For a whole-table continuation the scope is the universal key set, and the whole-table continuation result below is the form to use. This definition is strict: omitting, reordering, or duplicating an *in-scope* event violates $ContSeg(H, K, f, f', \delta)$, invalidating the continuation. Padding the segment with *out-of-scope* events also fails $ContSeg$, even though the resulting equality constrains only keys in K .

11.2 Continuation Across Frontiers

The primary theorem of this section continues a virtual cut to a later frontier. Let H be a wellformed source history. If σ is a virtual cut at frontier f on scope K , and δ is the continuation segment for $(f, f']$ on K , then σ followed by δ (sequence concatenation, written $\sigma @ \delta$) is a virtual cut at the later frontier f' on the same scope:

$$\begin{aligned}
& H \text{ wellformed} \wedge \text{VCS}(\sigma, K, f, H) \\
& \wedge \text{ContSeg}(H, K, f, f', \delta) \\
& \implies \text{VCS}(\sigma @ \delta, K, f', H).
\end{aligned}$$

Theorem Summary.

- **Status.** Machine-checked under the displayed assumptions.
- **Claim.** Appending the valid continuation segment for $(f, f']$ advances a virtual cut at f to a virtual cut at f' on the same scope.
- **Assumptions.** H is a wellformed source history, σ is a virtual cut at f on K , and δ is exactly the continuation segment for $(f, f']$ on K .
- **Boundary.** The conclusion is a replay-against-source equality on K . It is not a whole-table claim, not a statement that a verifier accepts an extended certificate, and not a destination-apply claim.

The wellformedness condition is needed because the proof uses the source history's coordinate order to know that the events in $(f, f']$ are exactly the ones between the two frontiers. Without monotonicity, events outside the interval could interleave.

Proof idea. The argument is pointwise on a key $k \in K$. Replay of a sequence at k is fixed by the latest replay event for k . If the continuation segment δ contains an event for k , then the latest k -event of $\sigma @ \delta$ lies in δ and is the change-data reading of the latest source event for k in $(f, f']$. A Layer 0 source-state interval lemma equates that event's effect with $\text{src}(b_0, H, f')$ at k . If δ contains no event for k , replay of $\sigma @ \delta$ at k equals replay of σ at k , which the hypothesis equates with $\text{src}(b_0, H, f)$ at k . The same interval lemma equates that with $\text{src}(b_0, H, f')$ at k , because no event for k in the interval means the source state for k does not change across it. The interval lemma (that src at f' is determined by src at f together with the source events in $(f, f']$) is the one new Layer 0 fact the continuation needs. It is proved from the source-coordinate order.

Whole-table continuation. The whole-table specialization (Section 10) delivers, under its claim-scope side conditions, the *unrestricted* equality $\text{apply}(\sigma) = \text{src}(b_0, H, f)$. Continuation specializes to that case. For a wellformed H , if $\text{apply}(\sigma) = \text{src}(b_0, H, f)$ holds on every key and δ is the continuation segment for $(f, f']$ on every key, then $\text{apply}(\sigma @ \delta) = \text{src}(b_0, H, f')$. This is the all-keys instance of the theorem above, machine-checked under the same assumptions. A *scoped* continuation cannot be promoted to a whole-table guarantee without this all-keys condition. A key the source inserts outside K within $(f, f']$ changes the table while leaving the scoped equality on K intact. Whole-table continuation requires the continuation segment for every key, just as whole-table specialization requires its claim-scope side condition.

11.3 Restriction to a Sub-Scope

A virtual cut also restricts downward along the scope. If σ is a virtual cut at frontier f on scope K , then it is a virtual cut at f on every sub-scope $K' \subseteq K$:

$$\text{VCS}(\sigma, K, f, H) \wedge K' \subseteq K \implies \text{VCS}(\sigma, K', f, H).$$

This is machine-checked. It holds for any sub-scope $K' \subseteq K$. Restriction is strictly one-directional. *Widening* a scope is unsound. For a key outside K the cut certifies nothing about that key's state at f , so there is no sound lemma extending a cut from K to a larger scope.

The same fact holds through a certificate's accessors. If C is a certificate with $\text{VC}(C, H)$ and $K' \subseteq \text{scope}(C)$, then the certified clean prefix is a virtual cut at $\text{frontier}(C)$ on the narrowed scope K' :

$$\begin{aligned}
& \text{VC}(C, H) \wedge K' \subseteq \text{scope}(C) \\
& \implies \text{VCS}(\text{prefix}(C), K', \text{frontier}(C), H).
\end{aligned}$$

This restricts the equality already established by a certificate to a smaller scope. Validating a restricted certificate for K' requires constructing and checking the corresponding evidence bundle, rather than following automatically from the restriction lemma alone.

11.4 Continuation Over an Accepted Certificate

Continuation composes with the certificate layer. Layer 3 (Section 9) proves that an accepted certificate, under accurate source observation, certifies a virtual cut at $\text{frontier}(C)$ on $\text{scope}(C)$. Appending a continuation segment advances that cut forward. Let H be a wellformed source history. If a certificate C is accepted under accurate source observation, and δ is the continuation segment for $(\text{frontier}(C), f']$ on $\text{scope}(C)$, then the certified clean prefix followed by δ is a virtual cut at f' on $\text{scope}(C)$:

$$\begin{aligned}
& \text{verify}(C, E) = \text{Accept} \wedge \text{FSO}(E, b_0, H, \text{raw}) \\
& \wedge H \text{ wellformed} \\
& \wedge \text{ContSeg}(H, \text{scope}(C), \text{frontier}(C), f', \delta) \\
& \implies \text{VCS}(\text{prefix}(C) @ \delta, \text{scope}(C), f', H).
\end{aligned}$$

Theorem Summary.

- **Status.** Machine-checked under checker obligations S1–S4 (Table 7), accurate source observation, source-history wellformedness, and exact continuation on $(\text{frontier}(C), f']$.
- **Claim.** An accepted certificate's virtual cut continues, by appending a valid continuation segment, to a virtual cut at a later frontier.

- **Assumptions.** Certificate acceptance under checker obligations S1–S4 (Table 7) and accurate source observation, a wellformed source history, and a continuation segment matching the source’s change data on $(frontier(C), f']$.
- **Boundary.** The conclusion is a replay equality on the sequence $prefix(C) @ \delta$. It is not a claim that the verifier accepts an extended certificate. Acceptance for the longer prefix remains an evidence-and-verifier obligation.

The continuation segment δ is itself an external observation of the source. Its accuracy with respect to H is an assumption that must be satisfied at deployment, exactly as the certificate’s own evidence is. The theorem composes the Layer 3 certificate theorem with continuation across frontiers. The composition leaves the trust assumptions for the certificate evidence and the continuation segment unchanged.

11.5 Ongoing Operation as a Sequence of Cuts

Continuation segments chain. For a wellformed source history, the continuation segment for $(f, f']$ followed by the continuation segment for $(f', f'']$ is the continuation segment for $(f, f'']$. Wellformedness is again required. Only a non-decreasing source history guarantees that the two coordinate ranges meet at f' without interleaving. Given a sequence of frontiers, the complete in-scope CDC segment over the combined interval can be split into segments at those frontiers. Appending the segments one at a time yields the same replay state as appending the combined segment.

DBLog’s replay-equivalence claim can therefore advance across frontiers. Each later cut consists of the original cut followed by the source changes committed in between.

Re-chunking and re-running. This property also governs repeated runs. Because a virtual cut at (K, f) depends solely on $src(b_0, H, f)$, any two replay sequences forming virtual cuts at (K, f) necessarily agree on K . A fresh wellformed run for the same scope and frontier reconstructs the same source state on that scope, independently of downstream replica state. Re-chunking is therefore an absolute, source-determined reconstruction of the per-key state (the regenerated replay prefix need not be identical, but replaying it yields the same source state), rather than a differential repair against a destination. Whether a destination then applies the regenerated prefix is a separate, sink-side question, which Section 12 leaves outside the current claims.

11.6 Scope of the Results

Each conclusion is a source-side *apply-against-src* equality. The continuation segment must be the source’s real change data on the interval, which is an External observation assumption with a Deployment obligation to retain that interval. These results prove no whole-table claim beyond the explicit all-keys instance, no acceptance of an extended or restricted certificate, and no destination-state convergence. Continuation

and restriction are machine-checked. Broader certificate-algebra extensions remain future work.

12 Limits on Sink-State Convergence

Layers 2 through 4 are source-side. They relate the certified replay prefix to the source at the certified frontier. They do not establish the state of a downstream destination.

Three destination properties must be distinguished. An *ACK* records receipt. A *durable audit log* persists what was received. *Applied state* is the state reached by the destination’s tables.

12.1 Receipt ACK

A destination ACK confirms that data was received. It does not guarantee that the destination applied that data, applied it in order, executed it idempotently, processed it without duplicates, or converged to the source state at the certified frontier. Receipt could support a future stream-correctness result, but it does not establish sink-state correctness.

12.2 Durable Audit-Log Persistence

A destination may durably record the certified clean prefix together with a run id, scope, frontier, or digest. This gives an auditable record of what arrived. Durable receipt is still stream-level evidence, not state-level evidence. A log can contain the right events while the destination table has not applied them or has applied them under different semantics.

12.3 Applied-State Evidence

To prove destination state convergence, a future layer would need a sink model with ordering, idempotence, fencing, visible apply barriers, and a way to observe that the destination state after the barrier corresponds to applying the certified clean prefix. This extension remains future work.

Even that future theorem would be scoped. Using *sinkState* as a placeholder for a future sink-state accessor, the target statement has the following form:

$$sinkState \upharpoonright scope(C) = src(b_0, H, frontier(C)) \upharpoonright scope(C).$$

Whole-table sink convergence would additionally need the whole-table side conditions from Section 10. Certificate acceptance alone does not guarantee that the destination database is correct. Establishing sink-state convergence requires both the source-side replay guarantee and an explicit sink execution contract.

A virtual cut relates replay to the source. An ACK establishes receipt, while an audit log establishes persistence. Destination-state guarantees require a fenced, idempotent apply contract and visible barrier evidence.

13 Related Work

The closest work comes from change-data-capture platforms, distributed cuts and frontiers, stream-processing watermarks, database recovery, and formal certificates. We compare these areas by concept and use named systems as deployment context.

13.1 DBLog Mechanism Lineage and Production Adopters

The DBLog mechanism was introduced in the December 2019 Netflix Tech Blog post [1] and described operationally in the 2020 DBLog paper [2]. It scans a table in primary-key chunks bracketed by log watermarks, discarding chunk rows whose keys overlap with CDC events anywhere within the window and emitting the remaining rows after the high watermark. We formalize this reconstructed state as a certified virtual cut. The model uses a canonical, non-overlapping chunk partition. The original mechanism’s retried or overlapping chunk reads must first be normalized into that canonical form. That normalizer is not developed here (Section 14.3).

Debezium [3] and Apache Flink CDC [4] include DBLog-style mechanisms in open-source CDC deployments. Debezium provides incremental snapshot modes, including signaling-table watermarks and read-only variants that use executed GTID sets for MySQL and MariaDB and transaction-snapshot information for PostgreSQL [5]. Apache Flink CDC combines Debezium-style connectors with Flink’s checkpointed streaming runtime [6], including parallel chunk readers [4].

The formal result does not transfer to these systems automatically. A deployment must still establish source-log retention, watermark or coordinate semantics, frontier ordering, and accurate source observation.

13.2 Cuts, Frontiers, Watermarks, and Checkpoints

The term “virtual cut” follows the distributed-systems language of logical time and consistent cuts. Lamport’s logical clocks [7], Chandy-Lamport snapshots [8], Mattern’s virtual time [9], Fidge’s vector clocks [10], and Pu’s on-the-fly consistent reading [11] are the nearest theoretical neighbors.

A DBLog virtual cut is not a global consistent cut over all processes. It is an extensional replay-equality statement. Replaying a certified prefix yields the source state at a frontier on a key scope. Ordering is governed by source-history coordinates instead of a vector-clock model of inter-process channels. Each cut is point-in-time, fixed to one frontier, but the cuts chain. The source-side continuation result (Section 11) advances a cut from one frontier to a later one, so a DBLog run corresponds to a sequence of source-side virtual cuts rather than a single one.

Stream-processing systems use adjacent terms. The Dataflow model [12] popularized event time and watermarks. Naiad [13] uses frontiers as progress markers. Flink’s asynchronous snapshots [6] checkpoint operator state for recovery. DBLog’s watermarks are source evidence around chunk reads, and *frontier(C)* is the source-history boundary for replay equality. It is not a stream-processing checkpoint of operator state, and it is not an exactly-once processing claim.

13.3 CDC Infrastructure and Source-Log Semantics

Data ingestion, transport, and coordination systems such as LinkedIn Databus [14], Facebook Wormhole [15], Kafka [16], Kafka Connect JDBC [17], and ZooKeeper [18] provide infrastructure for copying, transporting, and coordinating data. Kafka Connect JDBC provides polling-based table ingestion. These systems do not provide the certificates described in this paper.

Source-database documentation for MySQL binary logs and GTIDs [19], PostgreSQL logical decoding [20], PostgreSQL logical replication [21], and MariaDB GTIDs [22] identifies the source-log ordering and row-event semantics that a deployment must establish for accurate source observation.

Database-native and cloud CDC systems include PostgreSQL logical replication [21], CockroachDB changefeeds [23], AWS DMS [24], GCP Datastream [25], Oracle GoldenGate [26], Striim [27], Materialize’s PostgreSQL ingestion [28], Databricks Lakeflow [29], and Microsoft SQL Server CDC [30]. These systems provide components or workflows for the broader “initial load plus change stream” family. Our result adds a finite replay-equivalence certificate for the DBLog form of this pattern, under explicit source-observation assumptions.

CockroachDB resolved timestamps provide a production analogue of a source frontier. PostgreSQL logical replication illustrates the initial-copy-plus-ordered-change-stream pattern. Neither is treated here as exporting the finite, scoped evidence required by a virtual-cut certificate.

13.4 Recovery, Snapshots, Isolation, and Live Backfill

A database snapshot may refer to a recovery snapshot, a multi-version concurrency control (MVCC) read view, or point-in-time recovery (PITR). ARIES [31] and PostgreSQL PITR [32] reconstruct database state from checkpoints and logs. CALC [33] shows that database systems can construct logically consistent checkpoints without conventional synchronous physical checkpointing. Our contribution is different. It provides a scoped, source-side replay-equivalence certificate for a DBLog backfill, whose clean replay prefix equals source state at a named frontier and can be advanced to later

frontiers. Berenson et al. [34], Adya et al. [35], and Oracle Flashback [36] belong to the transaction-isolation and as-of-read tradition.

DBLog is not a recovery algorithm, an online backup mechanism, an MVCC snapshot, or a snapshot-isolation transaction. It does not assert that all chunk reads happened at one source timestamp. It asserts replay equality at a chosen frontier on a chosen scope.

Online schema migration tools such as gh-ost [37], F1 online schema change [38], and lazy schema migration systems [39] share the operational pressure of copying or rewriting data while writes continue. Their overlap with DBLog is live backfill under concurrent writes. Our formal result specifically establishes source-side replay equality for a DBLog-style backfill and its frontier advancement, rather than schema-state transitions or table swaps.

13.5 Related Verification Approaches

The certificate layer follows the general idea of checkable evidence under explicit assumptions. Refinement mappings [40], proof-carrying code [41], translation validation [42], version certificate recovery [43], and VerIso [44] all help position the idea of a finite object that establishes correctness relative to a model. Large mechanized artifacts such as seL4 [45], CompCert [46], TLA+ [47], Verdi [48], IronFleet [49], and Alloy [50] are cited for methodological comparison, not as theorem sources.

A DBLog certificate is not self-authenticating proof-carrying code, not a sink refinement map, and not an isolation-level conformance certificate. It is checkable evidence that, under accurate source observation, establishes a wellformed DBLog run and therefore certifies a virtual cut.

13.6 Scope of the Comparison

“Snapshot-equivalent replay” is intended literally but narrowly. A DBLog virtual cut is not a physical snapshot read, not a database snapshot-isolation read view, not PITR, not a stream-processing checkpoint, not a product replication guarantee, and not sink-state correctness. The result is a snapshot-equivalent replay certificate that does not require one physical snapshot read.

Debezium, Apache Flink CDC, MySQL, PostgreSQL, MariaDB, and the other named systems provide mechanism or deployment context. Applying the theorem to a concrete deployment requires its source-side assumptions, evidence semantics, and checker obligations to be established.

14 Future Work

Section 11 proves continuation across frontiers and restriction to a sub-scope. The following extensions are not proved here.

14.1 Concrete Deployment Paths

Extending the model to Debezium’s read-only modes would replace source watermark writes with observed markers: executed GTID sets for MySQL and MariaDB, and transaction-snapshot information for PostgreSQL [5]. A future theorem would need a refinement showing that these markers establish the same clean-prefix obligations as written watermarks, under the corresponding source observation assumptions.

Apache Flink CDC parallel chunk execution would require a parallel chunk-plan model, per-worker recovery assumptions, and a theorem connecting checkpointed parallel execution to the same abstract wellformed run used here. The certificate would need enough evidence to recover the single clean prefix consumed by the proof.

SELECT-at-LSN and source-positioned read modes would replace low/high watermark intervals with a point coordinate at which a chunk read’s row image is known to hold. These modes are attractive because they can avoid source writes, but they need new source-side assumptions and a new verification model for point-coordinate validation.

14.2 Sink Extensions

Destination-side results require a separate sink model.

ACK-only stream correctness would study receipt of the certified stream prefix. An ACK can support a theorem about receipt, but not about applied destination state. Such a proof would need a sink-receipt model and a stream-prefix equality theorem.

Strong sink barrier convergence would establish applied destination state. It needs a separate fenced and idempotent sink contract, visible apply-barrier evidence, and a proof that the destination state after the barrier realizes the certified clean prefix. It is not a consequence of the accepted-certificate theorem or the whole-table specialization by itself.

14.3 Certificate Algebra and Model Extensions

We prove restriction to a sub-scope and continuation across later frontiers by valid CDC segments. Disjoint composition, concrete extended-certificate verification, and a raw overlap/retry normalizer remain open problems. The current Layer 1 model uses a strict canonical chunk partition. A future normalizer would take overlapping or retried raw chunk reads and produce a canonical strict-partition certificate plus residual obligations, then prove that acceptance after normalization preserves the same virtual-cut claim.

Some notation and layer-boundary choices also remain open. A future paper might expose $prefix(C, E)$ instead of $prefix(C)$, use a relation form such as $certifiesPrefix$, or move overlap/retry concepts into Layer 1 instead of treating them through certificate materialization. Those changes would alter

the public theorem interface and require an explicit revision.

15 Verification Methodology

The companion Isabelle/HOL development checks the source and replay model, wellformed-run soundness, accepted-certificate soundness, whole-table specialization, continuation, and restriction. Each theorem holds under the assumptions stated with it.

A deployment must establish the assumed source-log retention, coordinate semantics, chunk coverage, and accuracy of CDC and chunk observations. The paper explains the definitions and proof arguments. Isabelle/HOL checks the formal derivations under their assumptions.

Constructed models establish non-vacuity, including a non-trivial accepted-certificate instance. Counterexample fixtures illustrate failures when selected conditions are removed. Some conjuncts follow from the construction or other conditions, as documented in the formal development.

16 Conclusion

In this paper, we formalized how DBLog combines chunk reads with captured changes and proved the correctness of the resulting replay. DBLog copies a table or a selected key range by reading data in chunks while the source continues accepting writes. Because chunks are read at different source-log positions, the collected rows do not form a physical snapshot.

We formalized this reconstructed state through the concept of a **virtual cut**. A virtual cut establishes snapshot equivalence. Replaying the sequence of chunk observations and streaming change events produces the exact state of the source database at a designated commit-log frontier, for every key in scope. The cut is *virtual* because rows are read at different times without pausing updates or taking a physical snapshot. DBLog emits the events needed to reconstruct the source state downstream, without storing that state itself.

We prove that this equivalence holds for every protocol run satisfying the stated run conditions. We then lift this guarantee to inspectable certificates, showing that an independent checker can verify the cut directly from recorded log evidence. When chunk reads cover the full table, this per-key guarantee establishes whole-table snapshot equivalence, accounting for both baseline rows and concurrent writes. We also prove that appending subsequent CDC events advances an established cut forward to later frontiers, and that cuts remain valid when restricted to smaller scopes. All results are machine-checked in Isabelle/HOL.

A certificate can expose missing or inconsistent evidence, but it cannot prove that a connector’s observations accurately reflect the physical source database. Source-log retention,

coordinate semantics, frontier selection, and chunk-read accuracy remain deployment obligations. The theorems establish source-side replay equivalence. They do not assert downstream sink convergence or exactly-once delivery, leaving concrete sink engines to separate verification.

Formal Development and Tools

The companion Isabelle/HOL development machine-checks the definitions and theorems under their stated assumptions. It does not discharge the *Deployment obligations* or the *External observation assumption*, which a deployment must establish separately. The formal development is publicly archived as the Zenodo deposit [51] (<https://doi.org/10.5281/zenodo.21732790>). The archived development declares no axioms. Every witness and counterexample fixture is a constructed instance. The only type not introduced as a datatype or record is a conservative typedef for source coordinates.

AI assistance. During the preparation of this work the author used the generative AI assistants ChatGPT 5.5 (OpenAI) and Claude Opus 4.7 (Anthropic) to support drafting and revising the exposition and developing the Isabelle/HOL formalization. The author directed the research, reviewed and edited all resulting text, and checked every definition, theorem statement, and proof. The author is the sole author of this paper and takes full responsibility for its content. The AI assistants are tools, not authors.

References

- [1] Andreas Andreakis and Ioannis Papapanagiotou. DBLog: A generic change-data-capture framework. Netflix Tech Blog, <https://netflixtechblog.com/dblog-a-generic-change-data-capture-framework-69351fb9099b>, 2019.
- [2] Andreas Andreakis and Ioannis Papapanagiotou. DBLog: A watermark based change-data-capture framework, 2020. Preprint arXiv:2010.12597.
- [3] Jiri Pechanec. Incremental snapshots in Debezium. Debezium Blog, <https://debezium.io/blog/2021/10/07/incremental-snapshots/>, 2021.
- [4] Apache Flink Community. Flink CDC MySQL CDC connector: How incremental snapshot reading works. Apache Flink CDC documentation (release-3.6), <https://nightlies.apache.org/flink/flink-cdc-docs-release-3.6/docs/connectors/flink-sources/mysql-cdc/#how-incremental-snapshot-reading-works>, undated. Online documentation. Accessed 8 September 2026.
- [5] Debezium Community. Debezium connector documentation: Read-only incremental snapshots

- for MySQL, MariaDB, and PostgreSQL. Debezium reference documentation, version 3.6, <https://debezium.io/documentation/reference/3.6/connectors/mysql.html#mysql-read-only-incremental-snapshots>; <https://debezium.io/documentation/reference/3.6/connectors/mariadb.html#mariadb-read-only-incremental-snapshots>; <https://debezium.io/documentation/reference/3.6/connectors/postgresql.html#postgresql-read-only-incremental-snapshots>, undated. Online documentation. Accessed 8 September 2026.
- [6] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Lightweight asynchronous snapshots for distributed dataflows, 2015. Preprint arXiv:1506.08603.
 - [7] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
 - [8] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.
 - [9] Friedemann Mattern. Virtual time and global states of distributed systems. In *Parallel and Distributed Algorithms*, pages 215–226. North-Holland, 1989.
 - [10] Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference (ACSC)*, pages 56–66, 1988.
 - [11] Calton Pu. On-the-fly, incremental, consistent reading of entire databases. *Algorithmica*, 1(1–4):271–287, 1986. doi: 10.1007/BF01840448.
 - [12] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.
 - [13] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. Naiad: A timely dataflow system. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 439–455, 2013. doi: 10.1145/2517349.2522738.
 - [14] Shirshanka Das, Chavdar Botev, Kapil Surlaker, Bhaskar Ghosh, Balaji Varadarajan, Sunil Nagaraj, David Zhang, Lei Gao, Jemiah Westerman, Phanindra Ganti, Boris Shkolnik, Sajid Topiwala, Alexander Pachev, Naveen Somasundaram, and Subbu Subramaniam. All aboard the Databus! LinkedIn’s scalable consistent change data capture platform. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*, pages 18:1–18:14, 2012. doi: 10.1145/2391229.2391247.
 - [15] Yogeshwer Sharma, Philippe Ajoux, Petchean Ang, David Callies, Abhishek Choudhary, Laurent Demailly, Thomas Fersch, Liat Atsmon Guz, Andrzej Kotulski, Sachin Kulkarni, Sanjeev Kumar, Harry C. Li, Jun Li, Evgeniy Makeev, Kowshik Prakasam, Robbert van Renesse, Sabyasachi Roy, Pratyush Seth, Yee Jiun Song, Benjamin Wester, Kaushik Veeraraghavan, and Peter Xie. Wormhole: Reliable pub-sub to support geo-replicated internet services. In *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 351–366, 2015.
 - [16] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.
 - [17] Confluent. JDBC source connector for Confluent Platform. <https://docs.confluent.io/kafka-connectors/jdbc/current/source-connector/overview.html>; https://docs.confluent.io/kafka-connectors/jdbc/current/source-connector/source_config_options.html, undated. Online documentation. Accessed 8 September 2026.
 - [18] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed. ZooKeeper: Wait-free coordination for internet-scale systems. In *Proceedings of the USENIX Annual Technical Conference*, pages 145–158, 2010.
 - [19] Oracle Corporation. MySQL 8.0 reference manual: The binary log. <https://dev.mysql.com/doc/refman/8.0/en/binary-log.html>; <https://dev.mysql.com/doc/refman/8.0/en/replication-gtids.html>, undated. Online documentation. Accessed 8 September 2026.
 - [20] PostgreSQL Global Development Group. PostgreSQL 18 documentation: Logical decoding. <https://www.postgresql.org/docs/18/logicaldecoding.html>; <https://www.postgresql.org/docs/18/logicaldecoding-output-plugin.html>, undated. Online documentation. Accessed 8 September 2026.
 - [21] PostgreSQL Global Development Group. PostgreSQL 18 documentation: Logical replication and architecture. <https://www.postgresql.org/docs/18/logical-replication.html>; <https://www.postgresql.org/docs/18/logical-replication-architecture.html>, undated. Online documentation. Accessed 8 September 2026.
 - [22] MariaDB Foundation. MariaDB knowledge base: Binary log; global transaction ID. <https://mariadb.com/kb/>

- en/binary-log/; <https://mariadb.com/docs/server/ha-and-performance/standard-replication/gtid>, undated. Online documentation. Accessed 8 September 2026.
- [23] Cockroach Labs. CockroachDB changefeeds: CREATE CHANGEFEED, changefeed messages, how does a changefeed work. <https://www.cockroachlabs.com/docs/stable/create-changefeed>; <https://www.cockroachlabs.com/docs/stable/changefeed-messages>; <https://www.cockroachlabs.com/docs/stable/how-does-a-changefeed-work>, undated. Online documentation. Accessed 8 September 2026.
- [24] Amazon Web Services. AWS database migration service: Full-load and CDC tasks. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Task.CDC.html; https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Tasks.CustomizingTasks.TaskSettings.FullLoad.html, undated. Online documentation. Accessed 8 September 2026.
- [25] Google Cloud. Datastream: Manage backfill for the objects of a stream. <https://docs.cloud.google.com/datastream/docs/manage-backfill-for-the-objects-of-a-stream>; <https://docs.cloud.google.com/datastream/docs/faq>; <https://docs.cloud.google.com/datastream/docs>, undated. Online documentation. Accessed 8 September 2026.
- [26] Oracle. Oracle GoldenGate online migration: Initial load plus CDC. <https://docs.oracle.com/en-us/iaas/database-migration/doc/set-oracle-goldengate-online-migrations.html>; <https://blogs.oracle.com/dataintegration/online-migration-leveraging-oracle-goldengate>, undated. Online documentation. Accessed 8 September 2026.
- [27] Striim. Striim: Initial-load and CDC replication. <https://www.striim.com/docs/en/getting-started.html>; <https://www.cockroachlabs.com/docs/stable/striim>, undated. Online documentation. Accessed 8 September 2026.
- [28] Materialize. Materialize PostgreSQL CDC source: Ingest data. <https://materialize.com/docs/ingest-data/postgres/>; <https://materialize.com/docs/ingest-data/postgres/postgres-debezium/>, undated. Online documentation. Accessed 8 September 2026.
- [29] Databricks. Lakeflow declarative pipelines: Change data capture and snapshots / AUTO CDC APIs. <https://docs.databricks.com/aws/en/ldp/what-is-change-data-capture>; <https://docs.databricks.com/aws/en/ldp/cdc>, undated. Online documentation. Accessed 8 September 2026.
- [30] Microsoft. What is change data capture (CDC)? – SQL Server. <https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-data-capture-sql-server>, undated. Online documentation. Accessed 8 September 2026.
- [31] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770.
- [32] PostgreSQL Global Development Group. PostgreSQL 18 documentation: Continuous archiving and point-in-time recovery (PITR). <https://www.postgresql.org/docs/18/continuous-archiving.html>, undated. Online documentation. Accessed 8 September 2026.
- [33] Kun Ren, Thaddeus Diamond, Daniel J. Abadi, and Alexander Thomson. Low-overhead asynchronous checkpointing in main-memory database systems. In *Proceedings of the 2016 ACM SIGMOD International Conference on Management of Data*, pages 1539–1551, 2016. doi: 10.1145/2882903.2915966.
- [34] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. A critique of ANSI SQL isolation levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, pages 1–10, 1995. doi: 10.1145/223784.223785.
- [35] Atul Adya, Barbara Liskov, and Patrick O’Neil. Generalized isolation level definitions. In *Proceedings of the 16th International Conference on Data Engineering (ICDE)*, pages 67–78, 2000. doi: 10.1109/ICDE.2000.839388.
- [36] Oracle. Using Oracle Flashback technology: Flashback query / AS OF SCN. <https://docs.oracle.com/en/database/oracle/oracle-database/19/adfns/flashback.html>, undated. Online documentation. Accessed 8 September 2026.
- [37] GitHub. gh-ost: GitHub’s online schema-migration tool for MySQL. <https://github.com/github/gh-ost>; <https://github.blog/news-insights/company-news/gh-ost-github-s-online-migration-tool-for-mysql/>, 2016.
- [38] Ian Rae, Eric Rollins, Jeff Shute, Sukhdeep Sodhi, and Radek Vingralek. Online, asynchronous schema change in F1. *Proceedings of the VLDB Endowment*, 6(11): 1045–1056, 2013. doi: 10.14778/2536222.2536230.
- [39] Zhilin Zeng, Hui Li, Xiyue Gao, Hui Zhang, Huiquan Zhang, and Jiangtao Cui. SLSM: An efficient strategy for lazy schema migration on shared-nothing databases, 2024. Preprint arXiv:2404.03929.

- [40] Martín Abadi and Leslie Lamport. The existence of refinement mappings. *Theoretical Computer Science*, 82(2):253–284, 1991. doi: 10.1016/0304-3975(91)90224-P.
- [41] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119, 1997. doi: 10.1145/263699.263712.
- [42] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166, 1998. doi: 10.1007/BFb0054170.
- [43] Jack Clark, Alastair F. Donaldson, John Wickerson, and Manuel Rigger. Validating database system isolation level implementations with version certificate recovery. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, pages 754–768, 2024. doi: 10.1145/3627703.3650080.
- [44] Shabnam Ghasemirad, Si Liu, Christoph Sprenger, Luca Multazzu, and David A. Basin. VerIso: Verifiable isolation guarantees for database transactions. *Proceedings of the VLDB Endowment*, 18(5):1362–1375, 2025. doi: 10.14778/3718057.3718065.
- [45] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.
- [46] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [47] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.
- [48] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 357–368, 2015. doi: 10.1145/2737924.2737958.
- [49] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, pages 1–17, 2015. doi: 10.1145/2815400.2815428.
- [50] Daniel Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 11(2):256–290, 2002. doi: 10.1145/505145.505149.
- [51] Andreas Andreakis. DBLog_Virtual_Cuts: Isabelle/HOL formal development for “A Theoretical Study of DBLog”, 2026. URL <https://doi.org/10.5281/zenodo.21732790>. Software, BSD 3-Clause License.